&

GitHub

Workshop

Who are we?

Section for Health Data Science and AI
Centre for Health Data Science (HeaDS)
Data Scientists, SUND Data Lab


Thilde

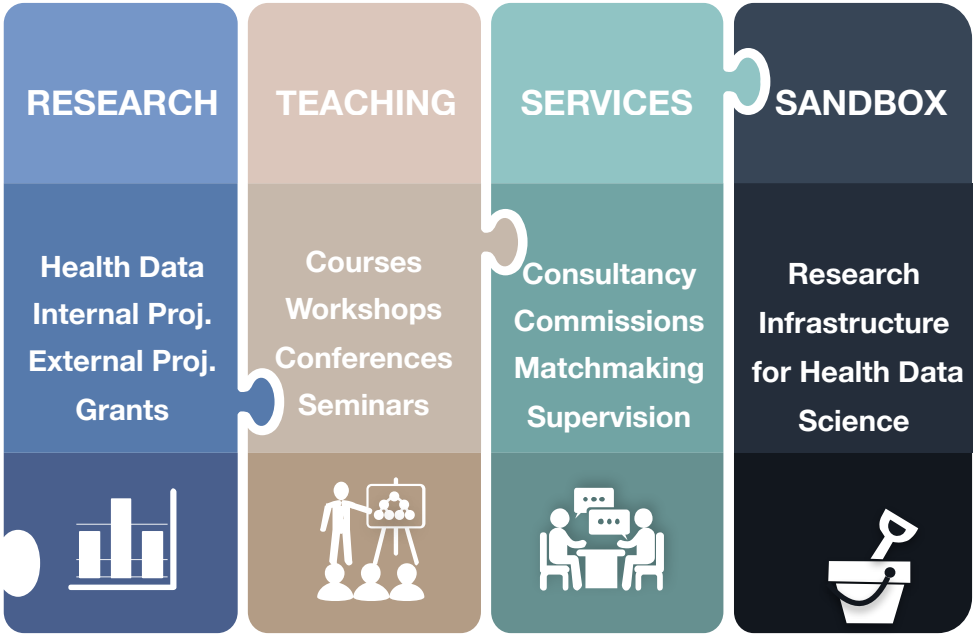

Diana


Suze

Center for Health Data Science

Scope:

- **Hub** for health data science research
- Conduct **health data science research**
- Develop National **HDS Sandbox**
- Host the **SUND Data Lab**

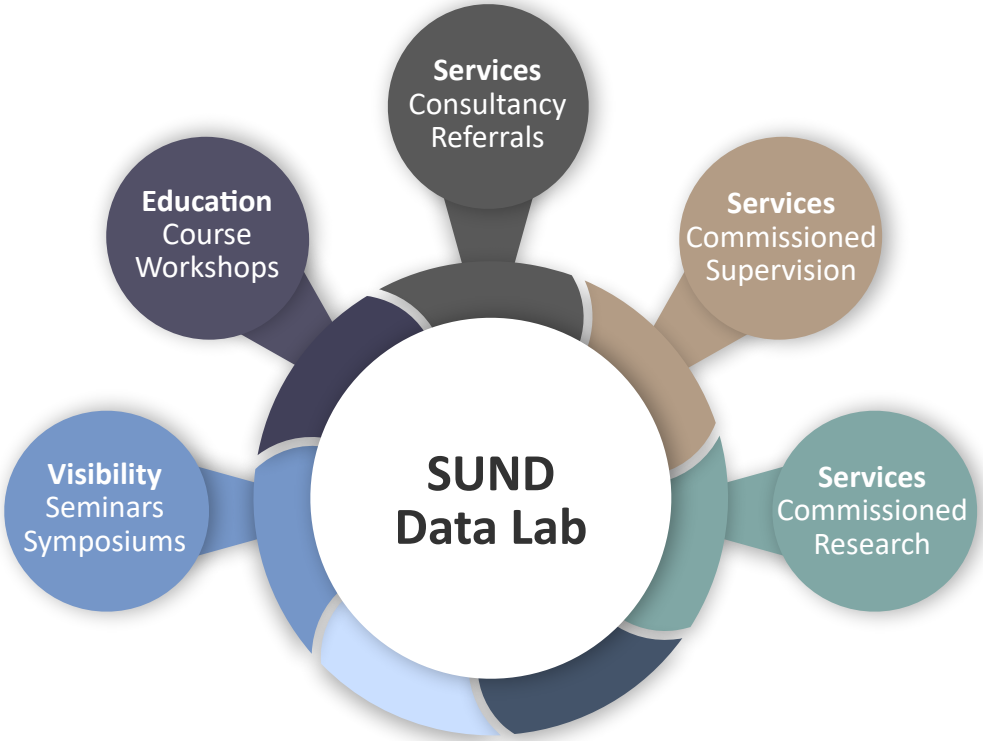


https://heads.ku.dk/

SUND DATA LAB

Website - https://heads.ku.dk
Email - datalab@sund.ku.dk

Follow us on **Twitter** at **@ucph_heads**!
Slack channel and **mailing list**
courses, conferences, research activities, job postings, etc. within the field of data science.



SUND DATA LAB

COURSES

Python

R

Bash

Git

HPC

GDPR

RNAseq

SKILLS

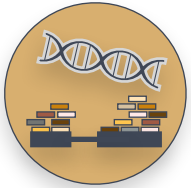
DEA

NW

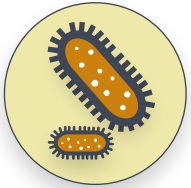
GESA

CLASS

RNA



BAC.



PROTEIN

&

GitHub

Workshop

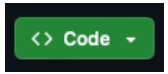
Let’s git started!

Program

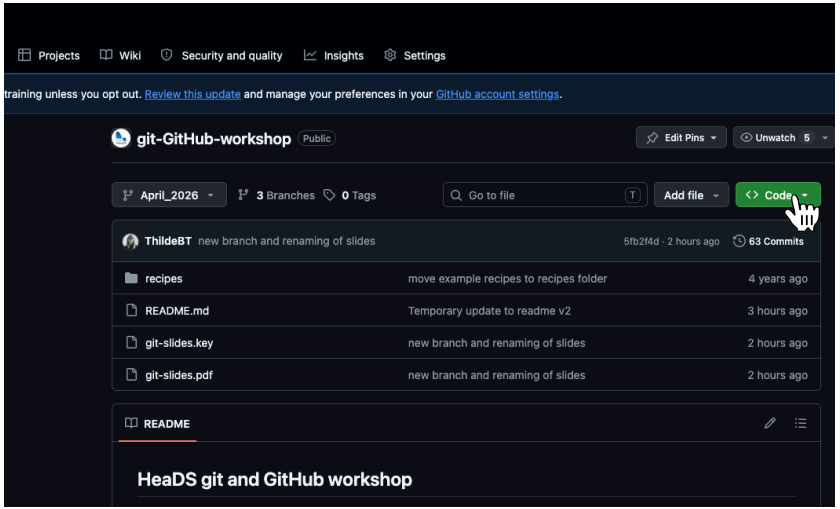
9.00-9.30:	Theoretical introduction
9.30-10.15:	First look at git/GitHub & commands
10.15-10.30:	Coffee Break
10.30-11.15:	Stage, commit & check history
11.15-12:00:	Branches, merging & conflicts
12.00-13.00:	Lunch break

13.00-13.30:	Reset and revert (back to the past)
13.40-14.30:	Interact with a remote repository
14.30-14.45:	Coffee Break
14.45-15:30:	Working together on GitHub
15.30-15:45:	Git & RStudio / Code editors
15.45-16.00:	Working with personal repos

How to get these slides

1. Go to: https://github.com/Center-for-Health-Data-Science/git-GitHub-workshop/tree/April_2026
2. Press 

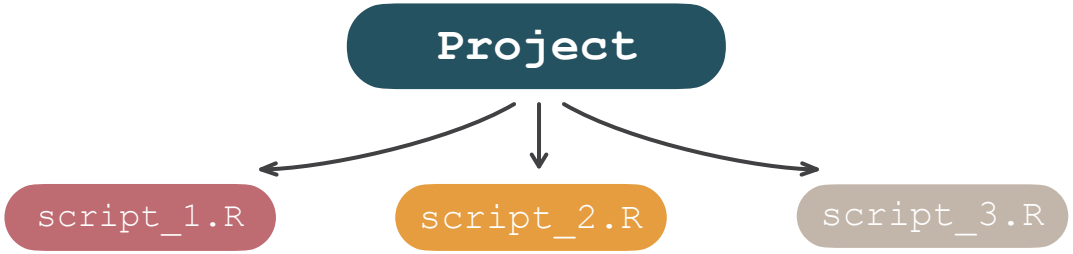
3. Download ZIP
4. Unzip & download on your computer



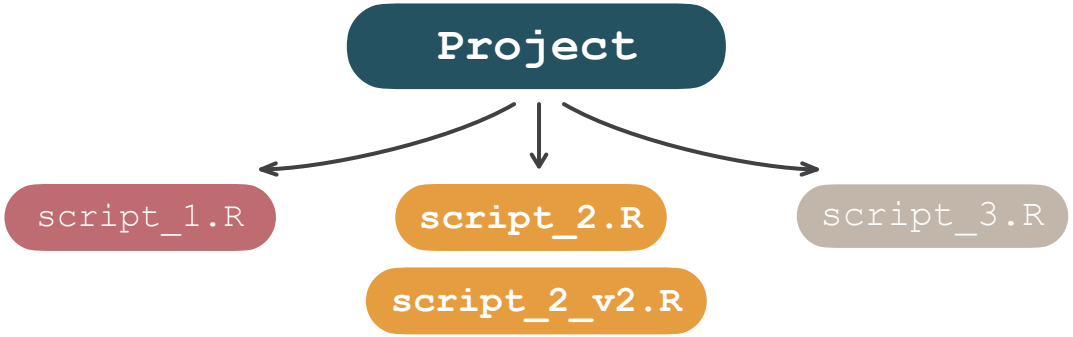
Justification

`Project`

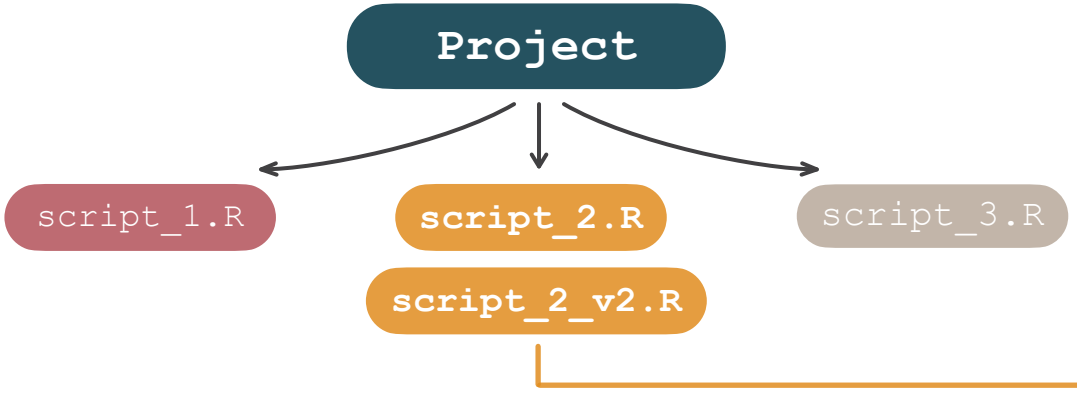
Justification



Justification



Justification

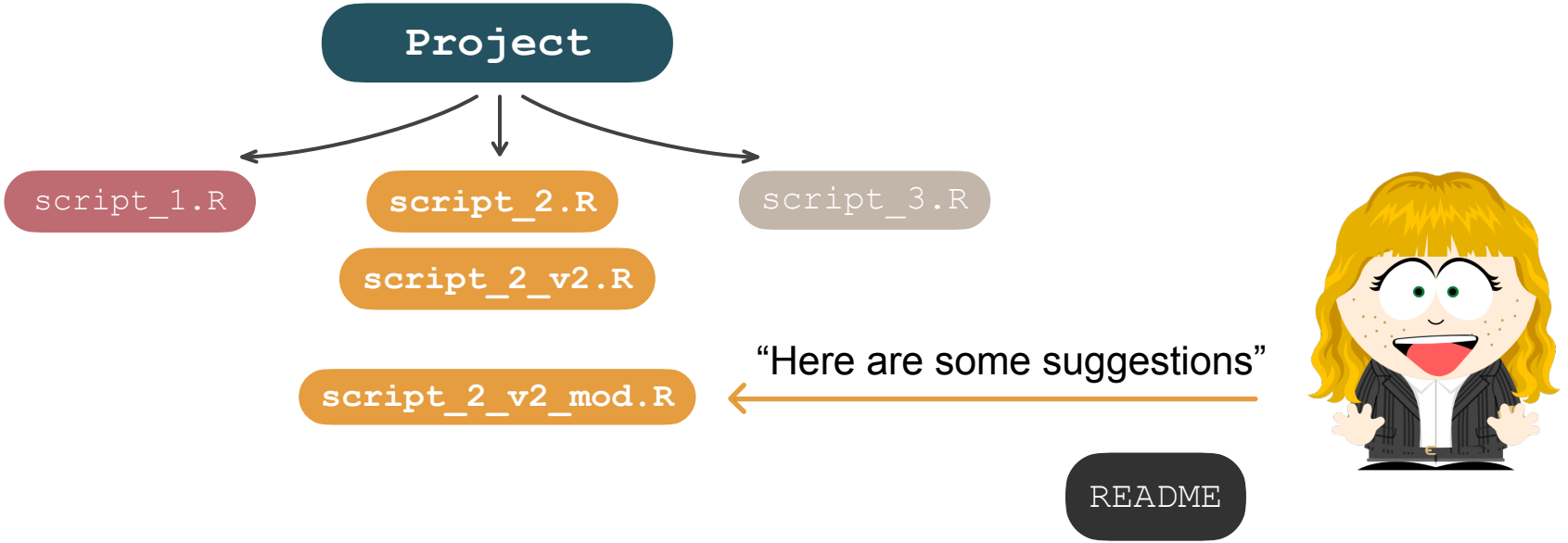


“Can I give it a try?”

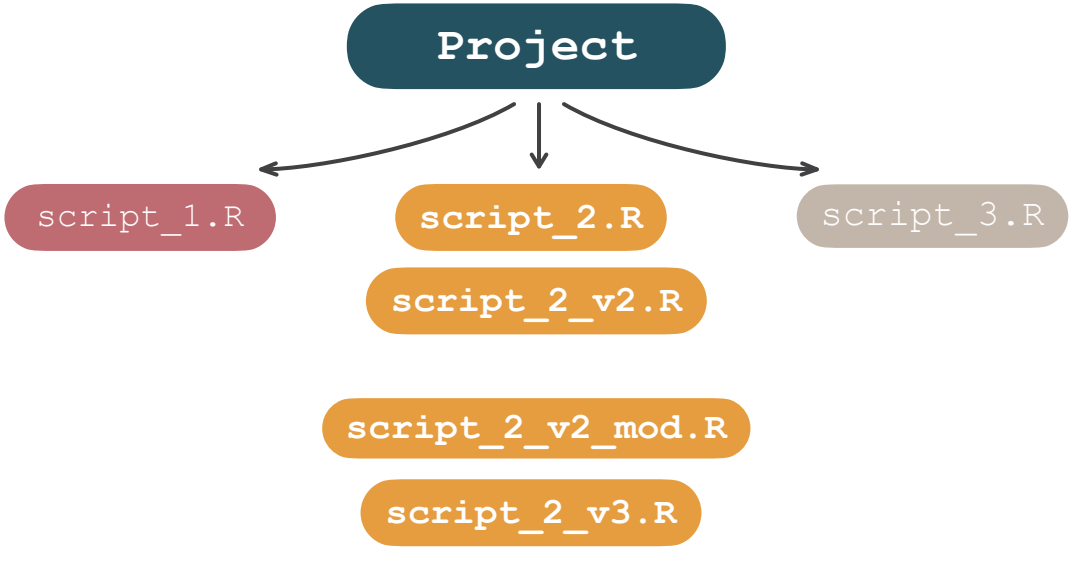
README

Descriptive file with info about script content & function

Justification



Justification



README

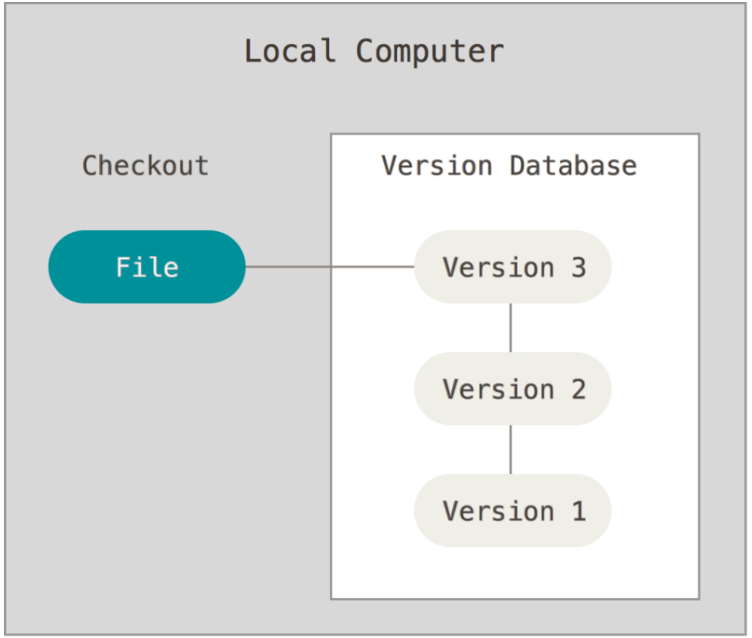
Considerations

Keeping track of things **manually** means:

- Significant overhead
  - Organization
  - Documentation
  - Communication
- Error-prone and time-consuming
- Makes collaboration difficult
- One year from now?

Version control systems

"Version control is a system that records changes to a file or set of files over time, so that we may remake specific older versions whenever we want (**history**)”



https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Benefits of VCS

Refer to and keep track of file versions

- When was “that” change introduced (backtracking)?
- Includes metadata (who, what, when, why)

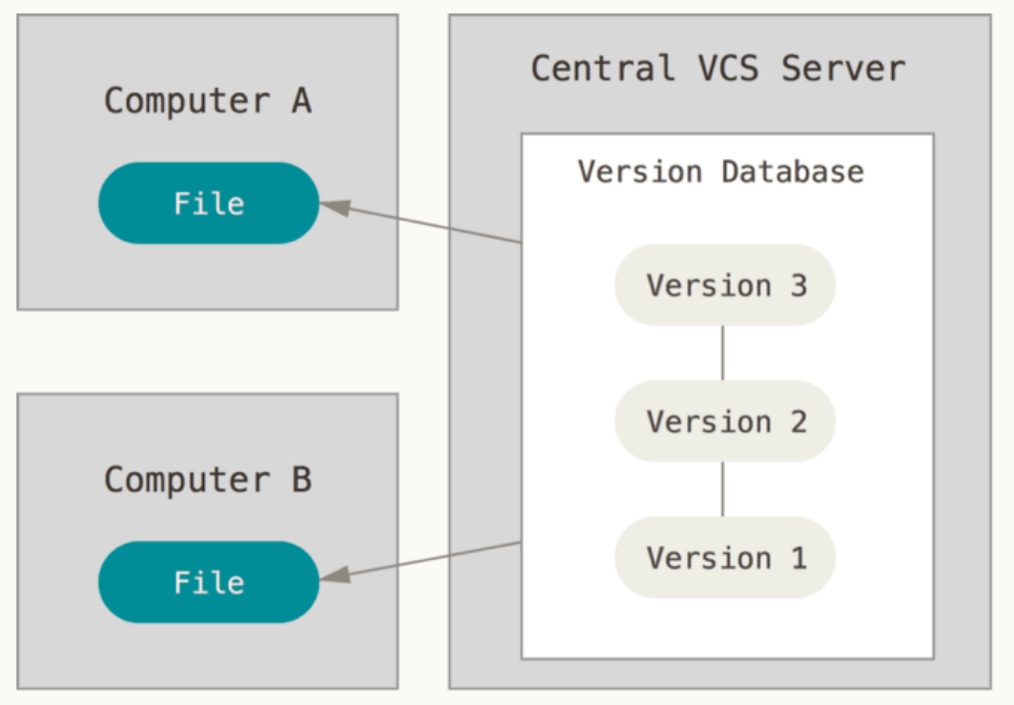
Organize and maintain long-term control over your code

- Experimenting is easy and organized
- Experiments are self-contained

Keep several versions of code at the same time, without clutter

- Keeps known working versions safe
- Able to easy retrieve old versions

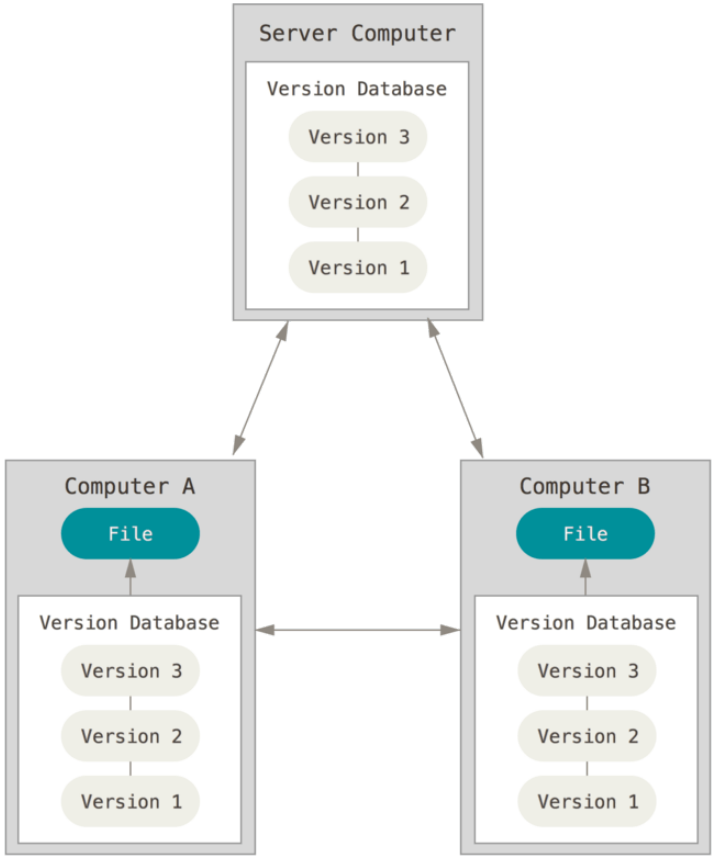
Centralized version control systems



https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Distributed version control systems (DVCS)

- No single point of failure
- Offline work is possible
- Performing most operations is fast
- The system keeps everyone synchronized…
- …while allowing for asynchronous work



https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

git and GitHub

Git is a DVCS software, created by Linus Torvalds for the management of the Linux kernel

Other clients exist.

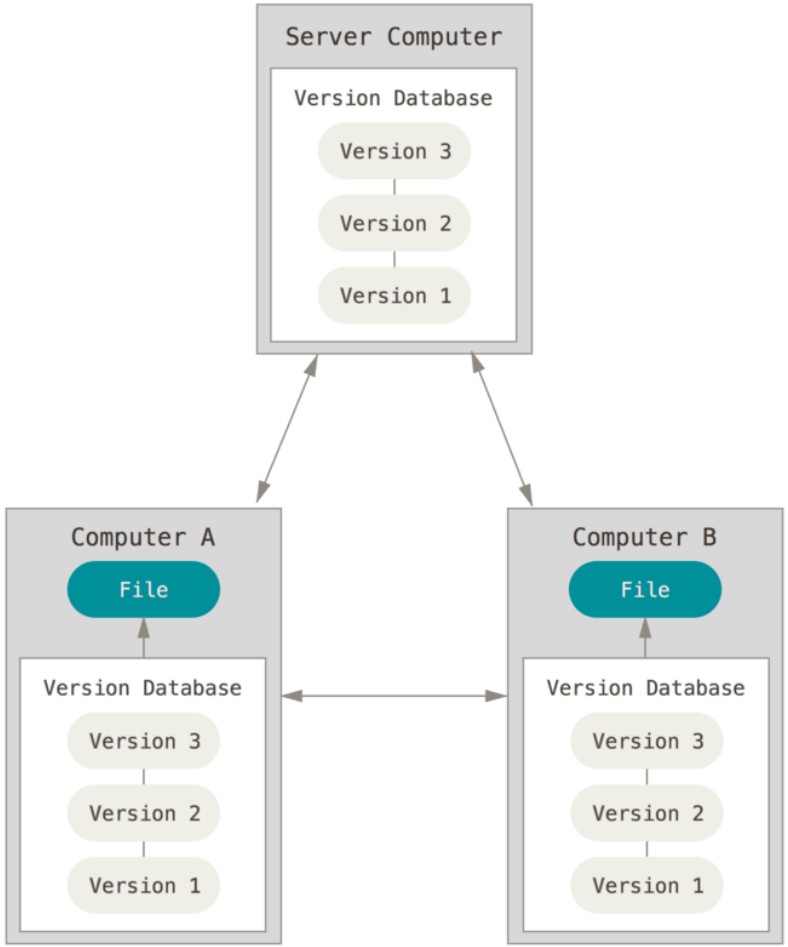
https://git-scm.com

GitHub is a web-based hosting service for version control that uses git

Other services exist.

https://github.com

git and GitHub



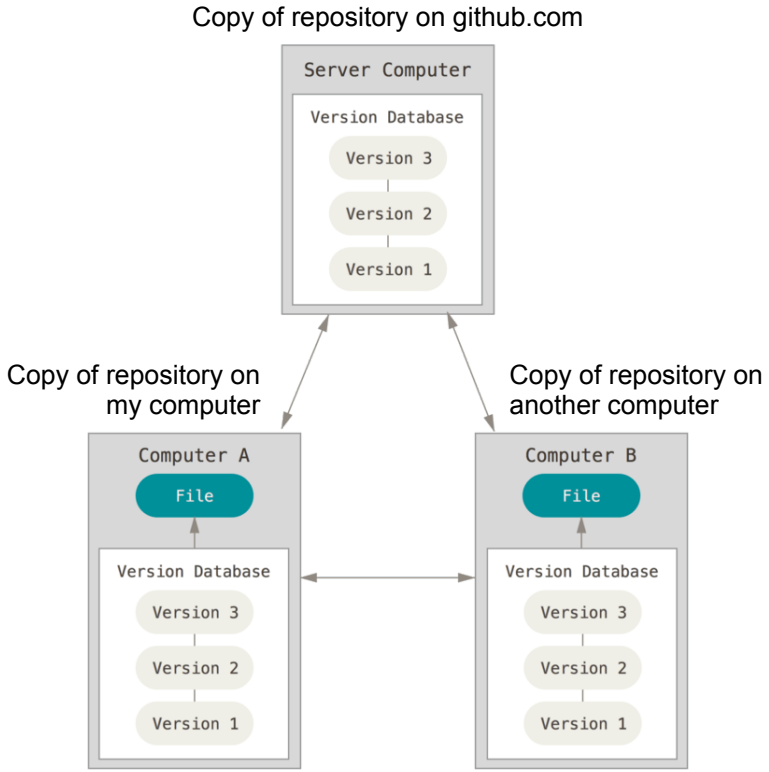
https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Useful concepts

Working directory (file/working tree):
All the files (organized in directories) that we want to keep track of, originating from a single directory.

Repository (repo)
Place in which changes to the working directory are recorded. Can be local or remote.
Several copies can exist.

Remote repository
Repository that is linked to our local repository.
Often a repository hosted on a site such as GitHub.



Why GitHub?

Visually browse and explore repositories, community aspect (e.g. issues, wiki), permission control, track changes

Public space: visibility, transparency, reproducibility

Easy to *collaborate* on the same repository

Remote repository (backup)

Easy to *contribute* to a repository (even if not a collaborator)

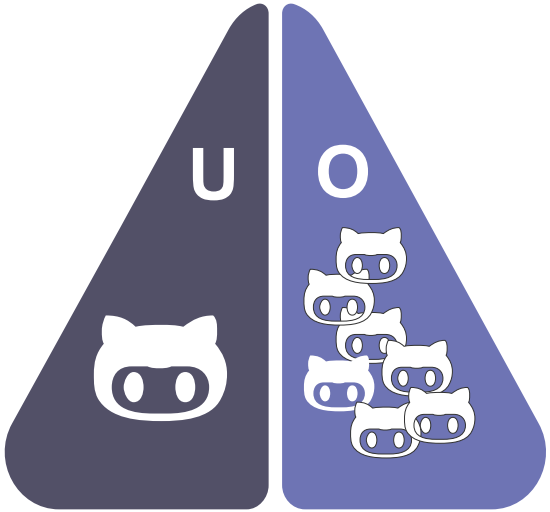
GitHub

GitHub is organized in

- **U** User accounts:

  Personal repositories that have a single owner.

- **O** Organizations:
  - Designed to better accommodate the needs of a team (e.g. lab).
  - One or multiple owners, and sophisticated access control systems to the repositories.



GitHub

GitHub repositories can be:

PUBLIC

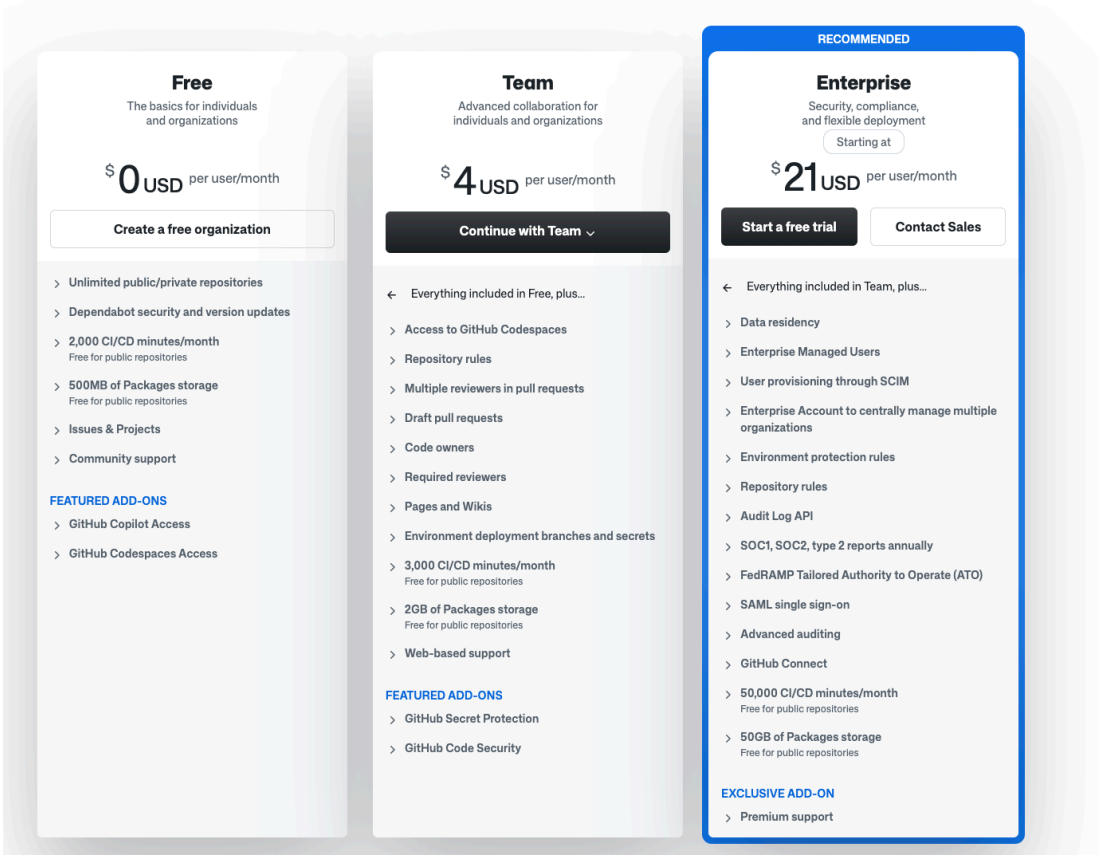
- Accessible to everyone
- Usually released with an open source license
- Still possible to have control over how many people interact with the repository.

PRIVATE

- Not accessible to the public.
- Accessible only from allowed accounts – requires password.

NOTE: Organizations have further options to limit and organize access to repositories

GitHub



Free

The basics for individuals and organizations

$0 USD per user/month

Create a free organization

- Unlimited public/private repositories
- Dependabot security and version updates
- 2,000 CI/CD minutes/month
  Free for public repositories
- 500MB of Packages storage
  Free for public repositories
- Issues & Projects
- Community support

FEATURED ADD-ONS

- GitHub Copilot Access
- GitHub Codespaces Access

Team

Advanced collaboration for individuals and organizations

$4 USD per user/month

Continue with Team

← Everything included in Free, plus...

- Access to GitHub Codespaces
- Repository rules
- Multiple reviewers in pull requests
- Draft pull requests
- Code owners
- Required reviewers
- Pages and Wikis
- Environment deployment branches and secrets
- 3,000 CI/CD minutes/month
  Free for public repositories
- 2GB of Packages storage
  Free for public repositories
- Web-based support

FEATURED ADD-ONS

- GitHub Secret Protection
- GitHub Code Security

RECOMMENDED

Enterprise

Security, compliance, and flexible deployment

Starting at

$21 USD per user/month

Start a free trial | Contact Sales

← Everything included in Team, plus...

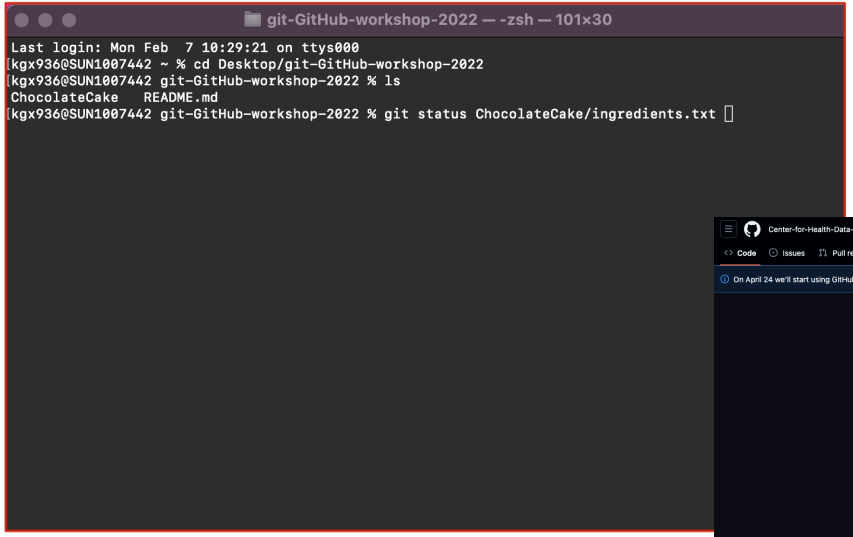
- Data residency
- Enterprise Managed Users
- User provisioning through SCIM
- Enterprise Account to centrally manage multiple organizations
- Environment protection rules
- Repository rules
- Audit Log API
- SOC1, SOC2, type 2 reports annually
- FedRAMP Tailored Authority to Operate (ATO)
- SAML single sign-on
- Advanced auditing
- GitHub Connect
- 50,000 CI/CD minutes/month
  Free for public repositories
- 50GB of Packages storage
  Free for public repositories

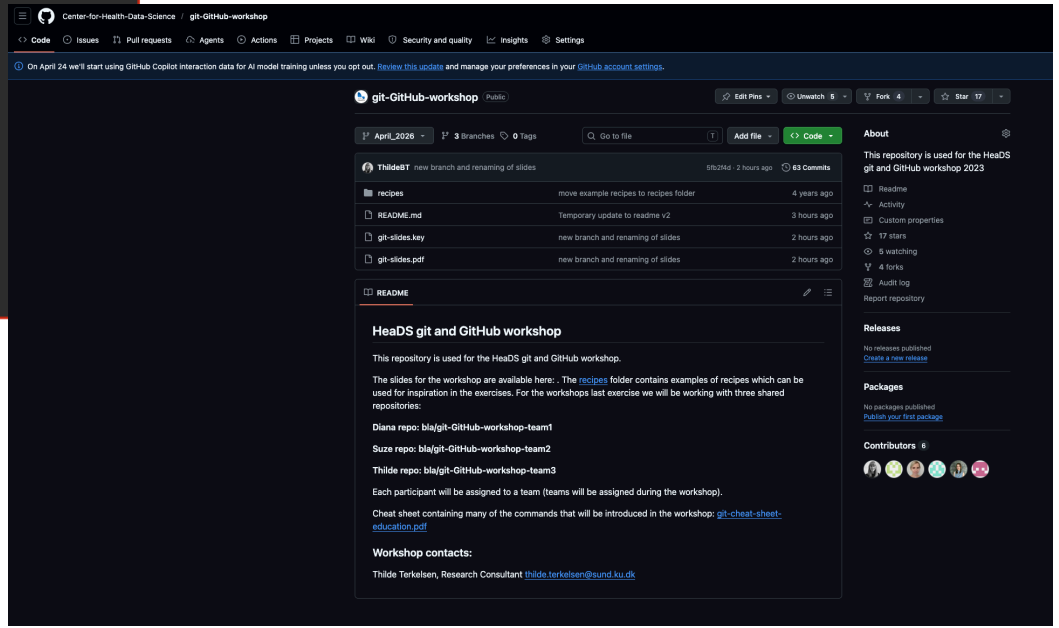
EXCLUSIVE ADD-ON

- Premium support

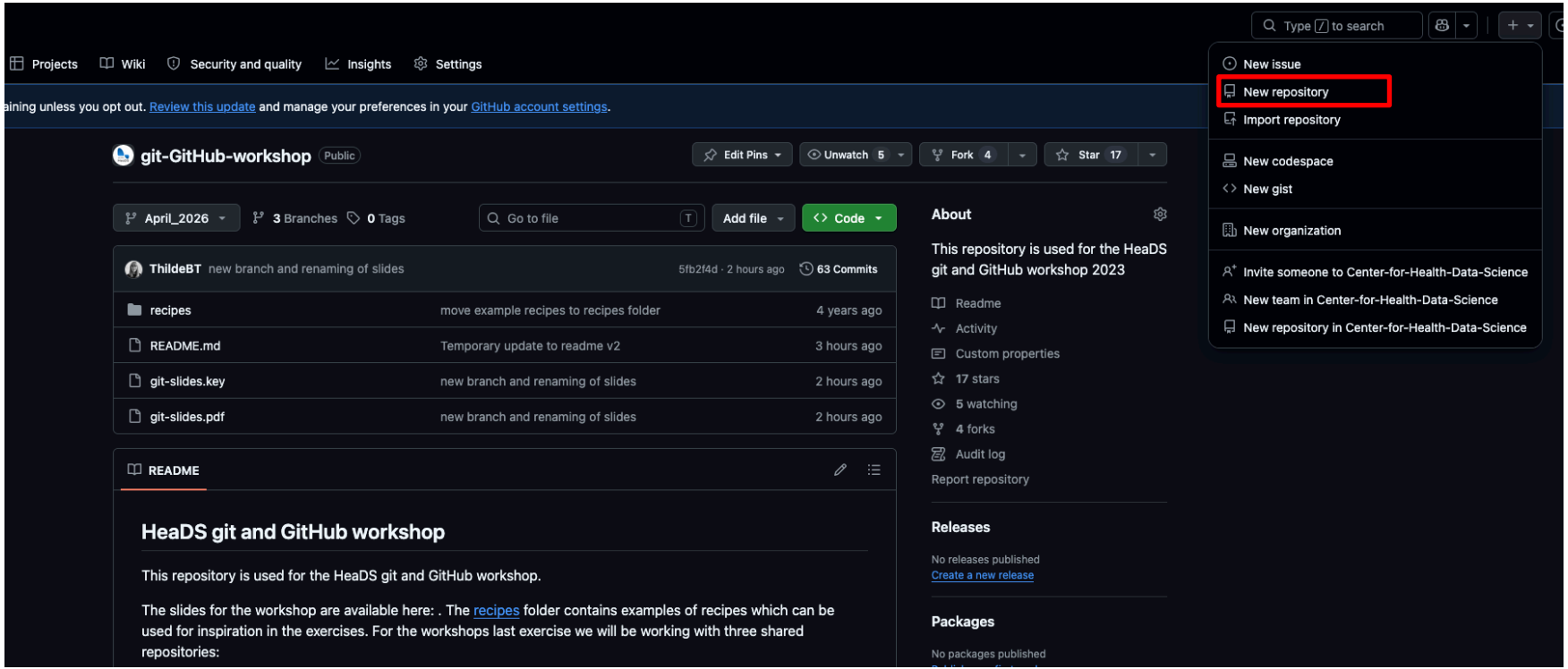
Academics can get free “Team” plan!

A look at git and GitHub

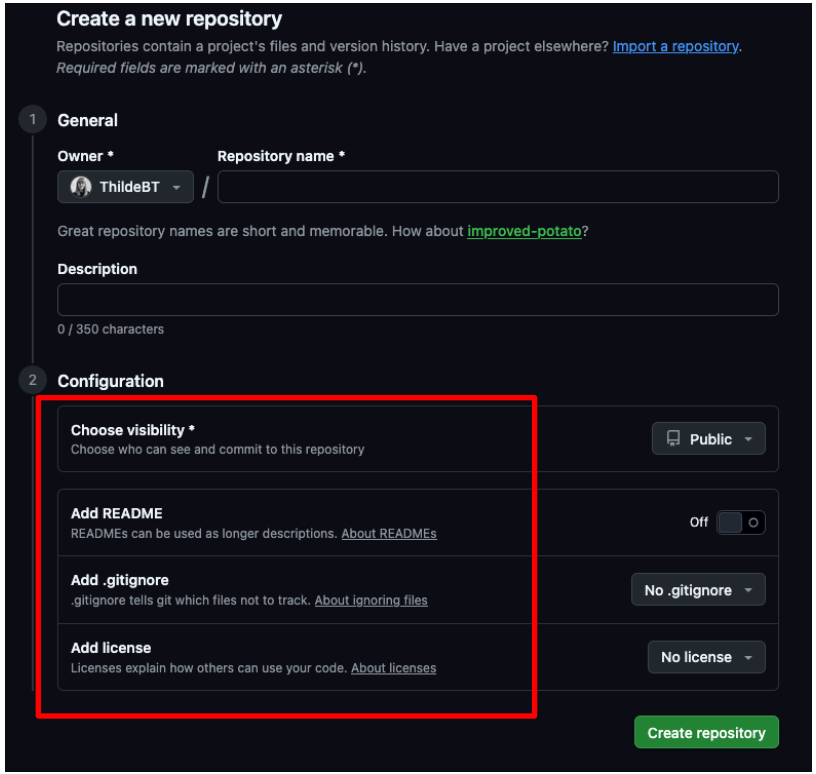




Let’s create a GitHub repo



Let’s create a GitHub repo



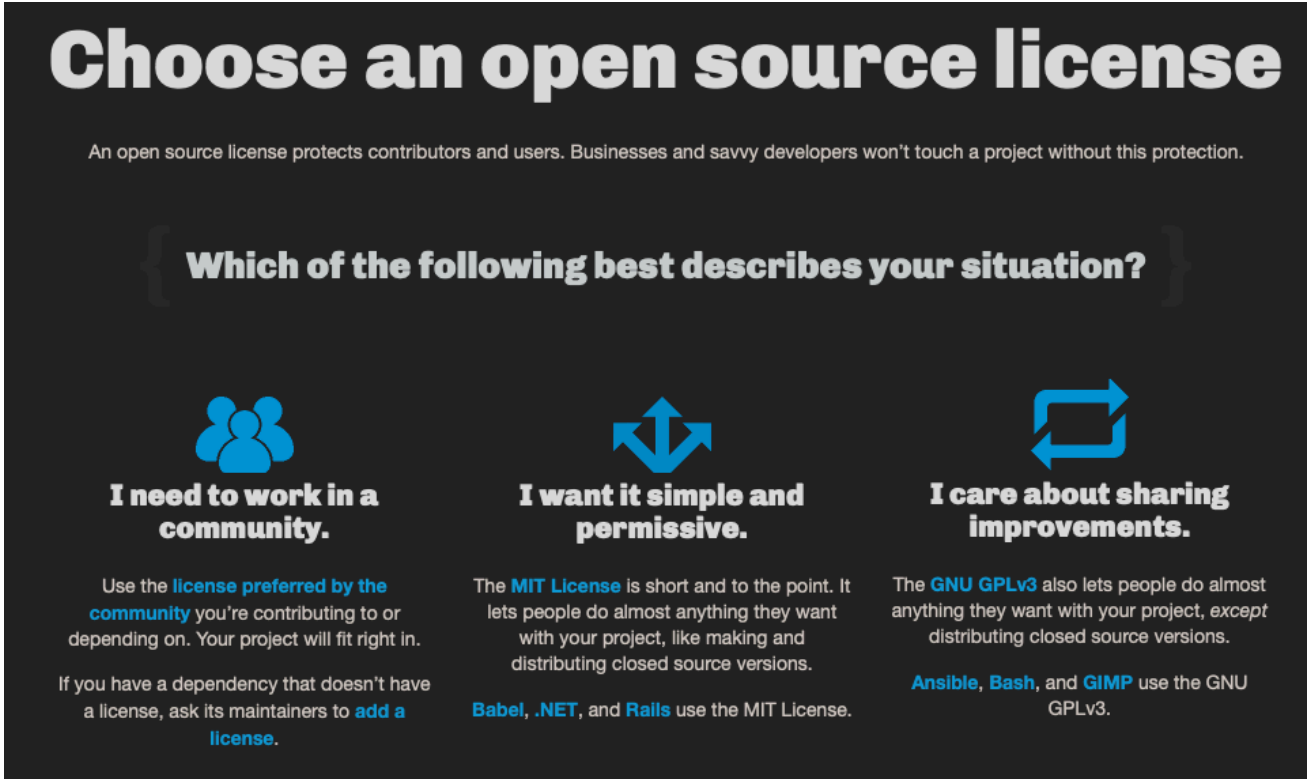
Visibility:
Public or Private Repository

README: File shown by default when someone visits the repository – good for an introduction to what the repository is about. Markdown format.

.gitignore: Hidden file containing specifications of files/types that should be ignored at all times by git. There are some default choices.

License: Creates a LICENSE file that contains the license of the content of your repository.

https://choosealicense.com



Commercial:

- MIT License
- Apache License

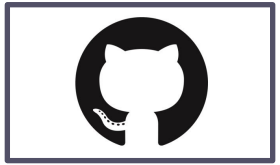
Open Source:

- GNU GPL v3.0

Resources

- Git Pro Book:
  https://git-scm.com/book/en/v2

- GitHub Skills:
  https://skills.github.com



- Git Tutorials:
  https://www.atlassian.com/git/tutorials

- Git Cheat Sheet:
  https://education.github.com/git-cheat-sheet-education.pdf

Program

9.00-9.30:	Theoretical introduction
9.30-10.15:	First look at git/GitHub & commands
10.15-10.30:	Coffee Break
10.30-11.15:	Stage, commit & check history
11.15-12:00:	Branches, merging & conflicts
12.00-13.00:	Lunch break

13.00-13.30:	Reset and revert (back to the past)
13.40-14.30:	Interact with a remote repository
14.30-14.45:	Coffee Break
14.45-15:30:	Working together on GitHub
15.30-15:45:	Git & RStudio / Code editors
15.45-16.00:	Working with personal repos

git syntax

We will use the terminal command `git`. Syntax:

```
git [command] [options] [args]
```

Args are usually names, files or directories.

Individual commands have help pages:

```
git [command] --help
```

Examples:

```
git add file.txt
git commit -m “changes were done”
git branch -d name
```

Getting started (first time user)

Configure git with your identity:

```
git config --global user.name [username]
git config --global user.email [email]
```

This is important if you want to associate your local changes with your GitHub account. If you want to keep your email private see this guide: *https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-personal-account-on-github/managing-email-preferences/setting-your-commit-email-address*

Change the default text editor used by git (for example to Nano):

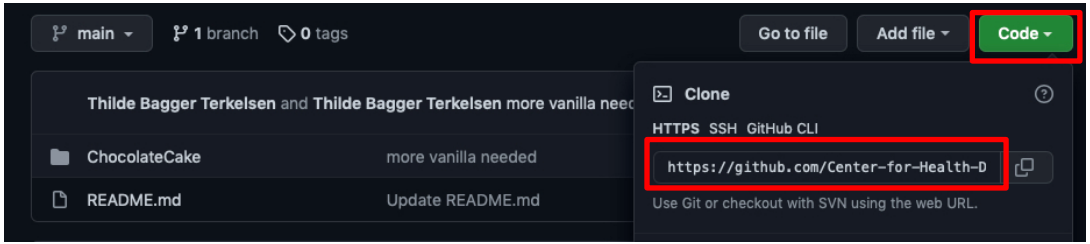
```
git config --global core.editor [text editor]
```

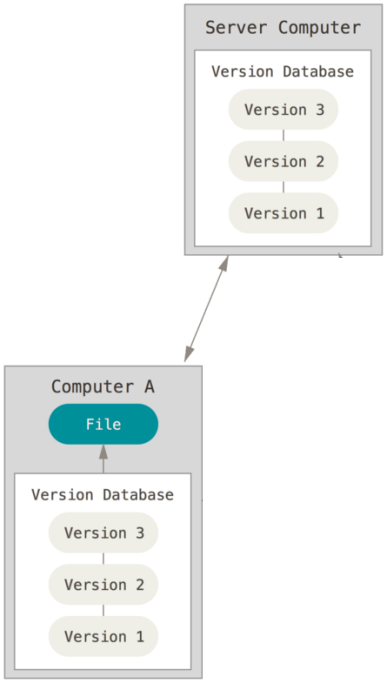
On a Windows system you must specify the full path to the text editor's executable file.

Cloning a GitHub repository

Create a local copy of a remote repository:

```
git clone [GitHub link]
cd [your repo]
```

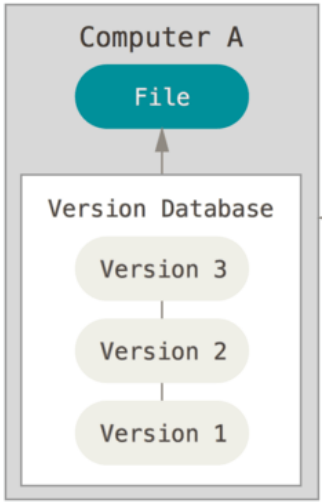




Creating a git repository

It is also possible to create a repository from scratch:

```
mkdir my-repo
cd my-repo
git init
```



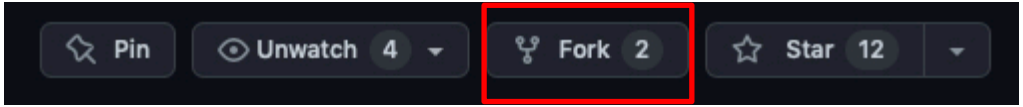
Note that in this case we would not have a remote.

Creating a repository on GitHub and cloning it ensures that:

- We have a remote repository
- Remote and local copy are linked

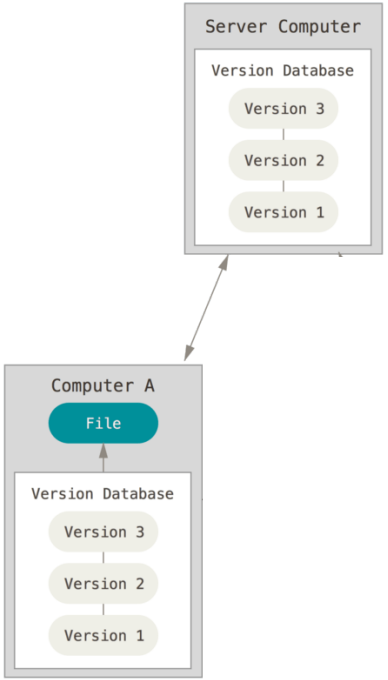
Forking a GitHub repository

Create a copy of someone else's repository on Github:



Forks are most commonly used to:

- Propose changes to another persons repository
- Use another persons repository as a starting point for own project



Checking status of a git repo

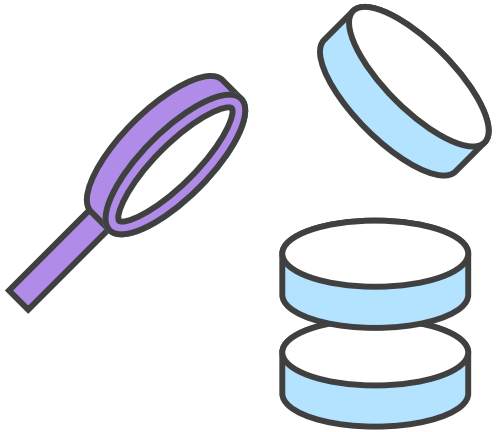
- Check the current state of the repository:

```
git status
```

  Gives, among other things, information on which files that have been modified.

- Check the name and location of the remote repository:

```
git remote -v
```



https://www.atlassian.com/git/tutorials/inspecting-a-repository

Exercise 1

Exercise
1

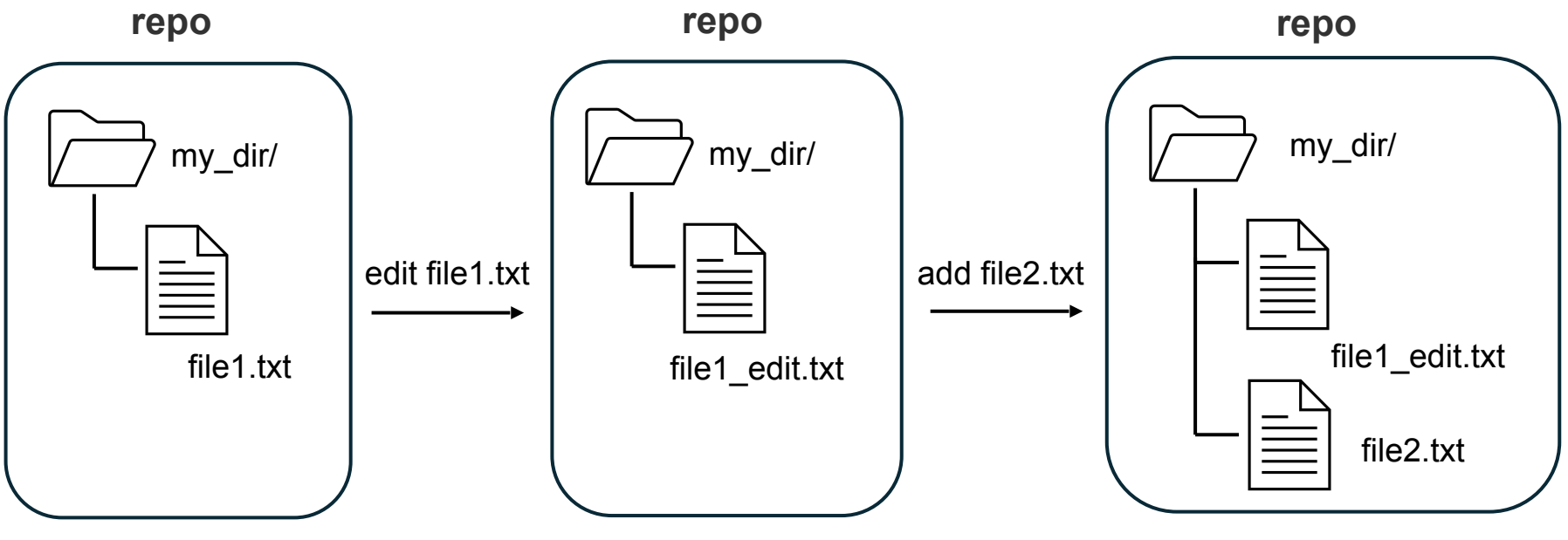
- 1. Configure git with username and email if you have not done so before.
- 2. Create a new public repository on GitHub (web interface not on your local terminal). This repo will hold your favorite cooking recipes. Remember to add a README file and choose a license.
- 3. Create a local copy of your GitHub repository by **cloning** it in a folder on your computer. Make note of the location of the directory that you choose to host your local copy.
- 4. Move to the local repository. Check the status of this repository (don't worry if all the information does not make sense yet). What do you think “working tree clean” means? Check the name and location of the remote repository.

Program

9.00-9.30:	Theoretical introduction
9.30-10.15:	First look at git/GitHub & commands
10.15-10.30:	Coffee Break
10.30-11.15:	Stage, commit & check history
11.15-12:00:	Branches, merging & conflicts
12.00-13.00:	Lunch break
13.00-13.30:	Reset and revert (back to the past)
13.40-14.30:	Interact with a remote repository
14.30-14.45:	Coffee Break
14.45-15:30:	Working together on GitHub
15.30-15:45:	Git & RStudio / Code editors
15.45-16.00:	Working with personal repos

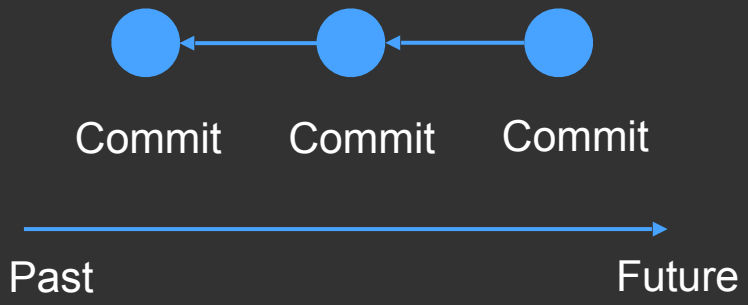
What is a commit?

A commit is a 'snapshot' of the current state of the project:



What is a commit?

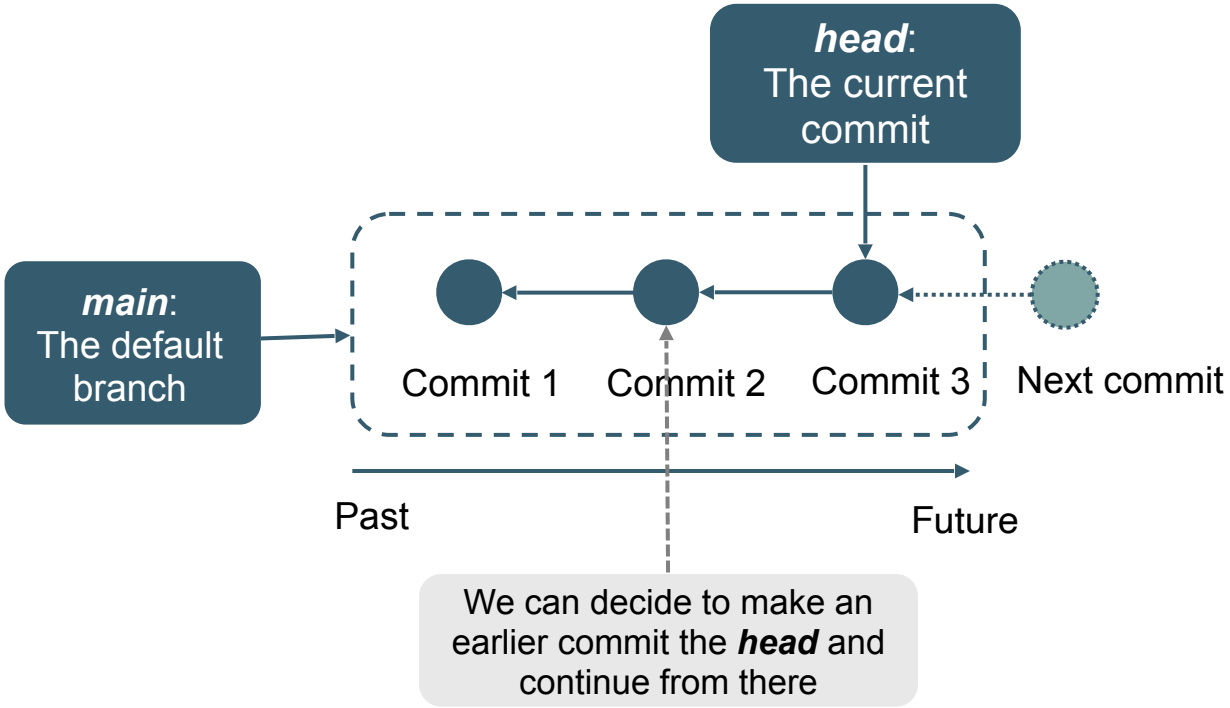
The commit is the atomic unit of a git working tree.
They are often illustrated with a circle in schematics:



When we show the commit history, we will often paint arrows pointing backwards

Commits do not store the actual files but the current state of the repo

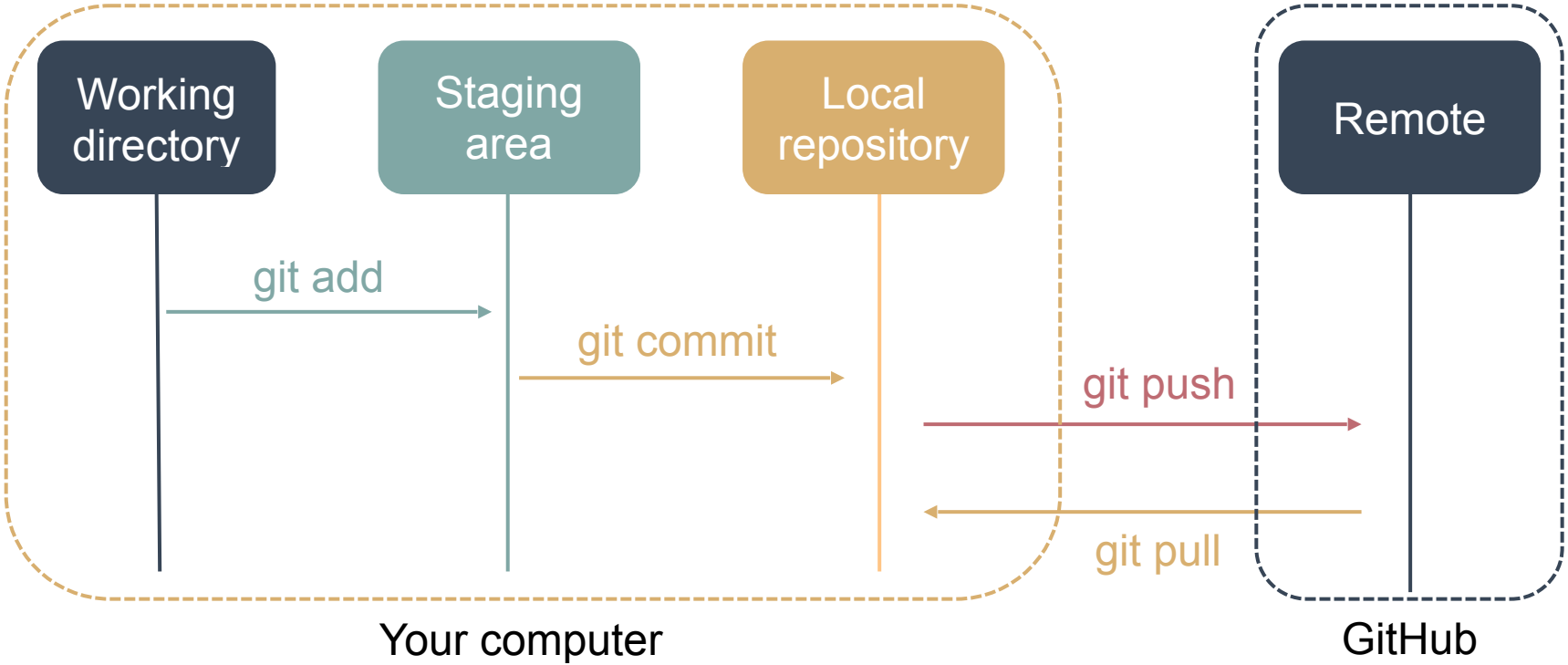
Useful concepts



New commits are built on / point back to the last ***head***

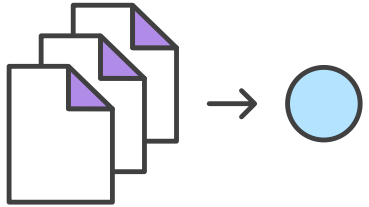
Commits have:
ID (=hash)
Message

What is staging



Adding files to stage

Files are added to stage using the command **`add`**



```
git add [filename]
```

- One can stage multiple files or even directories (if they contain files)
- Option `-A` stages all changes if no arguments are given
- More than one filename are allowed including wildcards

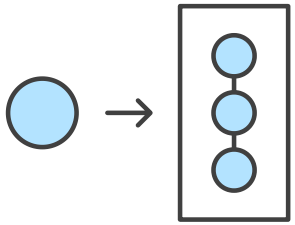
To remove or move a file (including staging the change) use:

```
git rm [filename]
git mv [filename]
```

https://www.atlassian.com/git/tutorials/saving-changes

First Commit

When you're happy with your changes and have added all the files to stage the next step is to commit the changes:



```
git commit
git commit -m "Commit Message"
```

Use `-a` to automatically stage files that have been modified or deleted. **New files you have not told git about are not affected.**

```
git commit -a -m "Commit Message"
```

https://www.atlassian.com/git/tutorials/saving-changes

Good practices

- **Commits** should be:
  - Atomic and self-contained (or WIP), but not excessively pedantic
  - Done often (try not to have gigantic blob commits)

- **Commit messages** should:
  - Always have a short comment (more descriptive than “bugfix”, “save”, etc.)
  - Written to be understandable by someone else >1 year down the line, including you

Checking history

Check the history of a repository:

```
git log
```

A few useful options:

- To see only the commits of a certain author

```
git log —author=[name]
```

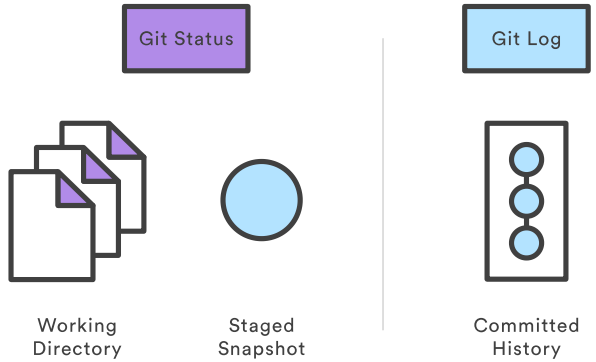
- To see a compact history where each commit is only one line

```
git log --oneline
```

- To see the history as a graph (useful with multiple branches):

```
git log --all --decorate --oneline --graph
```

press q to quit



https://www.atlassian.com/git/tutorials/inspecting-a-repository

Checking differences

Check

- not staged changes since the last commit:

```
git diff
```

- staged changes since the last commit:

```
git diff --cached
```

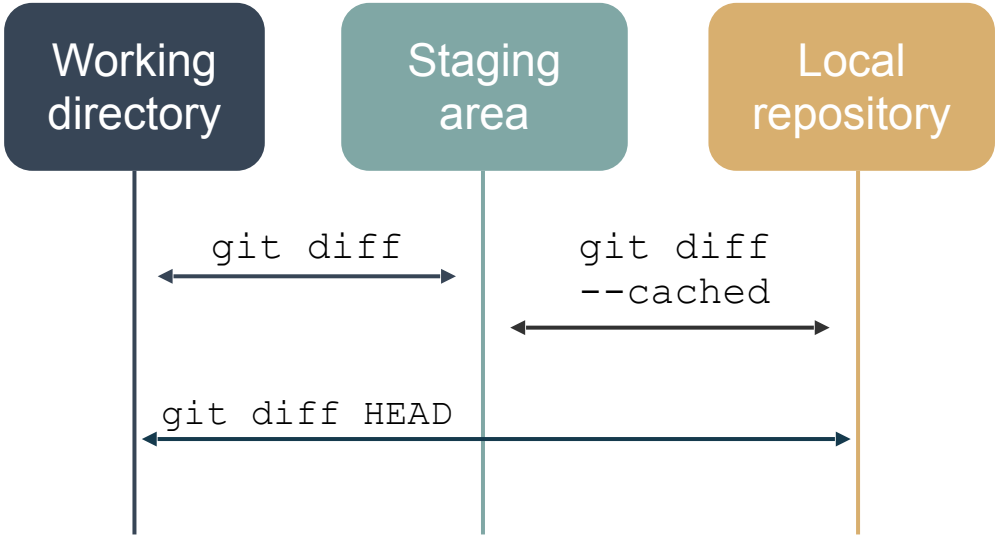
- differences between two commits:

```
git diff [ID1] [ID2]
```

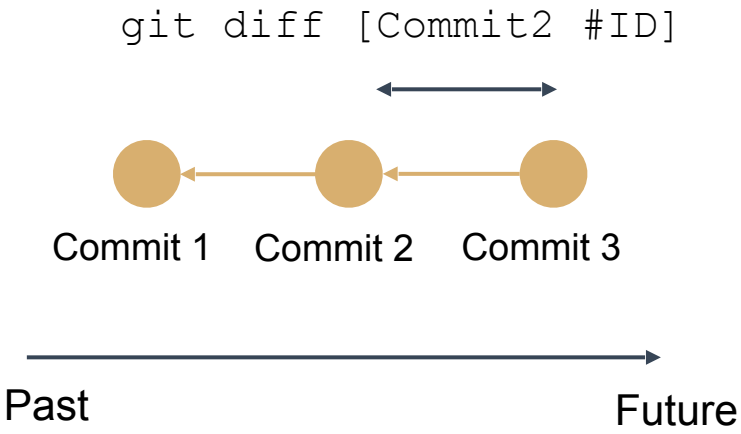
- differences between second last and last commit:

```
git diff HEAD^ HEAD
```

Checking differences



Differences between current state of project and last commit



Differences between commits

Useful concepts

Status: status of git repository

Commit:
Snapshot of a particular state of a working tree. Has an ID & message

Stage:
Selection of changes made on a repository to be added to a commit

origin = default name of original remote repository

main = default name of main branch (local and remote)

head = current commit (called ***detached head*** when not the tip of a branch)

Exercise 2

Exercise
2

- 1. Create two files named ingredients.txt and recipe.txt (can contain anything you would like) in the repository folder on your **local** computer (not GitHub!).
- 2. Check the status of the repository. **Stage** the new files (use `git add`) and check the status. **Commit** the staged files and check the status. What did you see change between each step when checking status?
- 3. Create a directory with the recipe name and move your files to it with a git command. Then create a commit with your changes.
- 4. Add or remove one step in *recipe.txt* and check the change **before staging** (use `git diff`). Stage and commit your change.

Exercise 2 cont.

Exercise
2

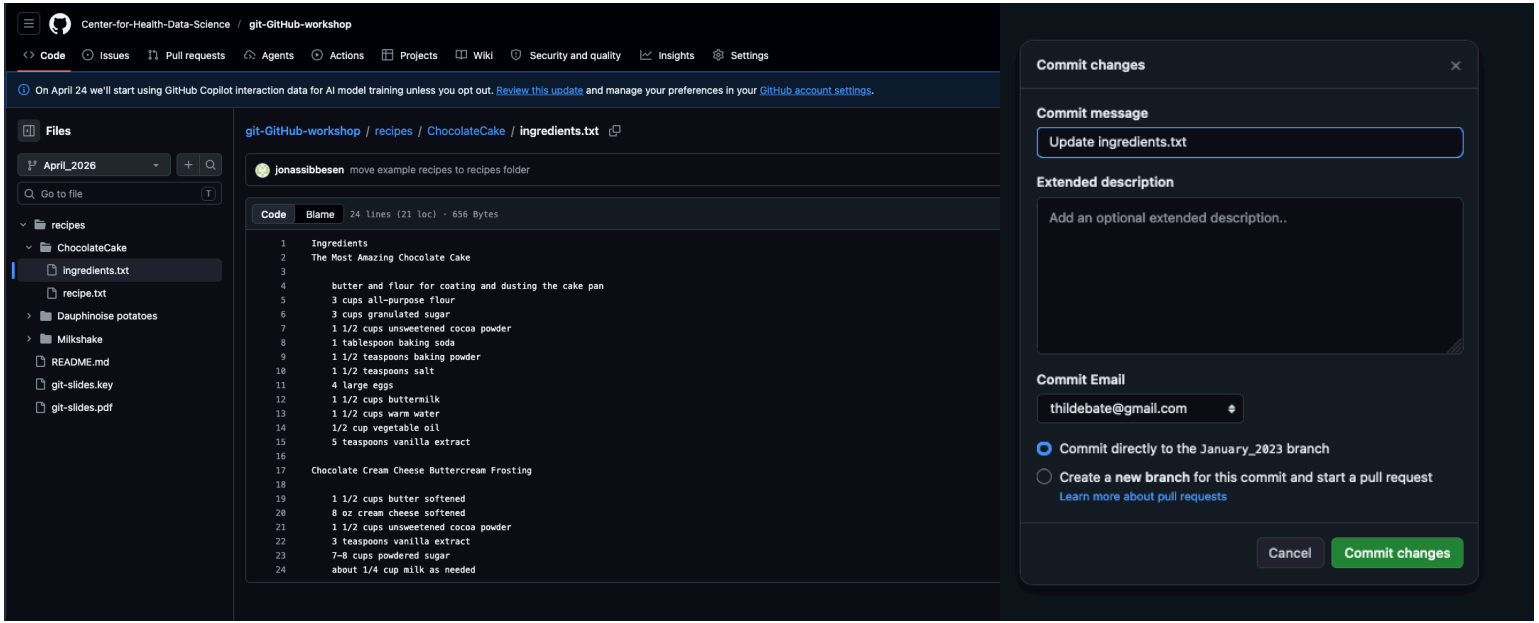
- 5. Check the commit history. Can you identify the author, ID, time and date of your commits? Try experimenting with the different ways of showing the history.
- 6. Check the difference between different commits. Can you identify the filename(s) and changes?

Program

9.00-9.30:	Theoretical introduction
9.30-10.15:	First look at git/GitHub & commands
10.15-10.30:	Coffee Break
10.30-11.15:	Stage, commit & check history
11.15-12:00:	Branches, merging & conflicts
12.00-13.00:	Lunch break
13.00-13.30:	Reset and revert (back to the past)
13.40-14.30:	Interact with a remote repository
14.30-14.45:	Coffee Break
14.45-15:30:	Working together on GitHub
15.30-15:45:	Git & RStudio / Code editors
15.45-16.00:	Working with personal repos

GitHub Intermezzo

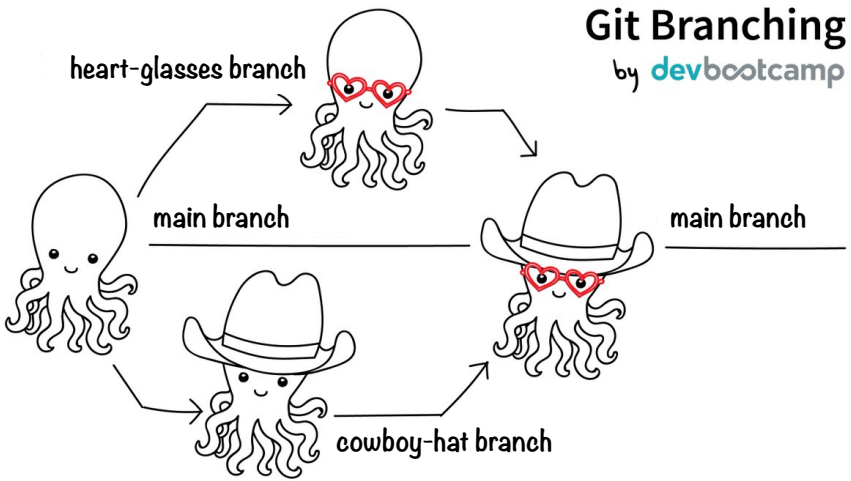
- You can edit files and commit changes directly from GitHub
- Not ideal when working with collaborators as it is more likely to create conflicts when local versions are merged/pushed



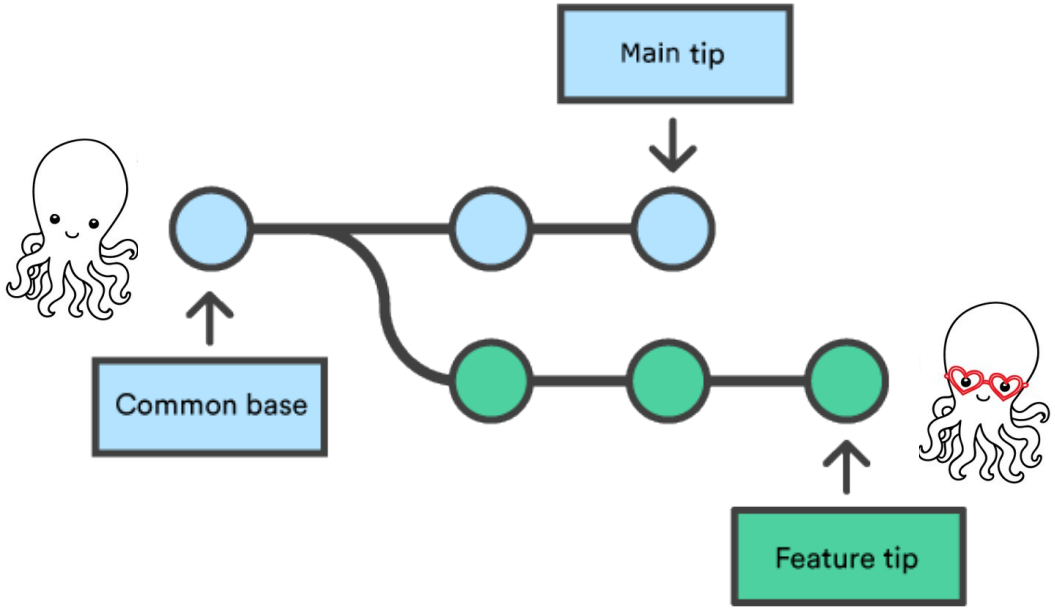
Branching

A **branch** represents an independent line/track of work on your repository.

- Keep a branch that is in working order (*main* branch).
- Isolate experiments from the *main* branch.
- Merge changes we like into the *main* branch.
- Work on more than one feature at the same time, independently.



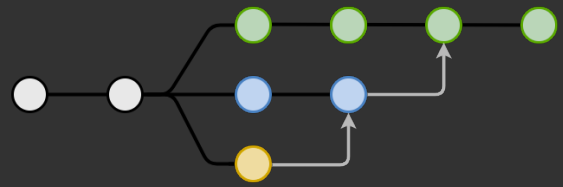
Branching



https://www.atlassian.com/git/tutorials/using-branches/git-merge

Branching

- List all the branches in your repository:
  `git branch -a`
- Create a new branch:
  `git branch [branch]`
- Switch to a branch:
  `git checkout [branch]`
  `git switch [branch]`
- Safe deletion: prevents deleting if branch has unmerged changes:
  `git branch -d [branch]`
- Force delete:
  `git branch -D [branch]`



Good practices

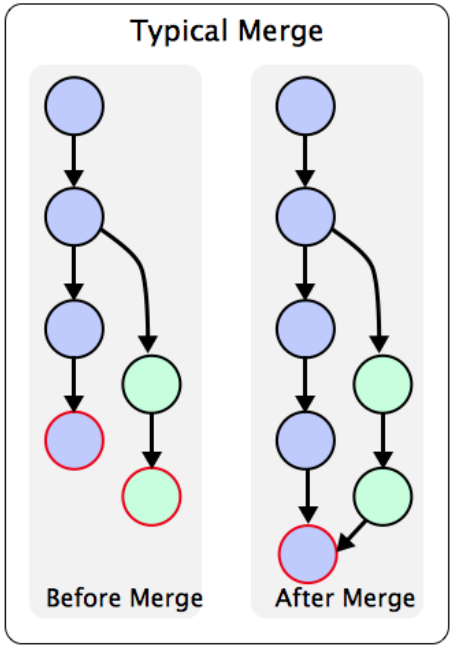
- **Branches** should:
  - Have descriptive names.
  - Be used every time you want to experiment (e.g. add a new feature or fix a bug).
  - Have a single purpose (e.g. one “feature”).
  - Derived from a sensible commit (often the latest commit on the *main* branch)

Merging

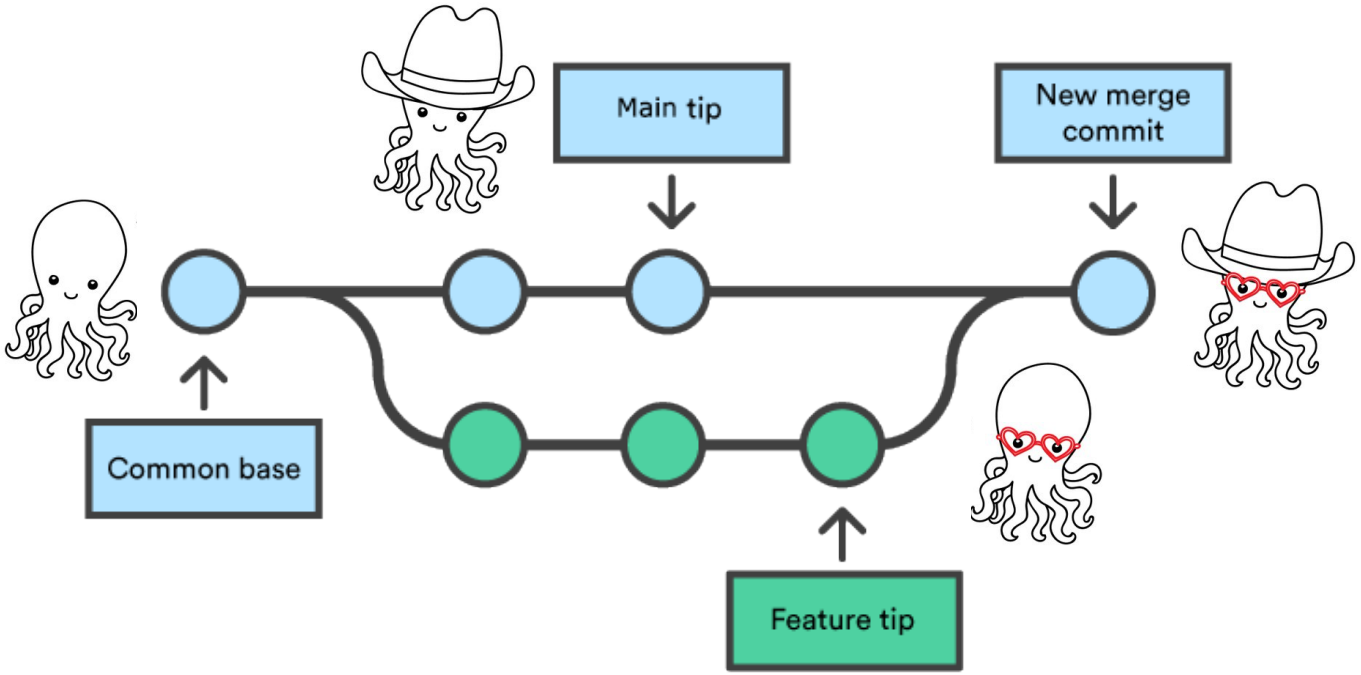
Merging incorporates changes made in another branch **into the current/active branch.**

- **Switch** to the branch you want to change (not the one you want to merge)
- **Merge** changes made in another branch **into the current/active branch.**

```
git checkout [active branch]
git merge [branch to merge]
```



Merging



https://www.atlassian.com/git/tutorials/using-branches/git-merge

Merging

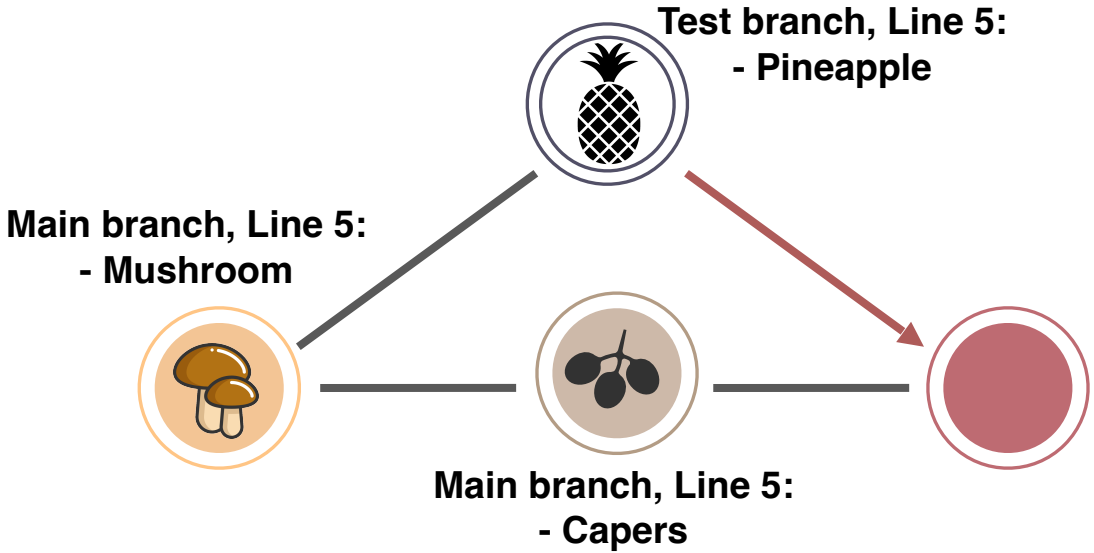
Two main cases for merging

We want to integrate a feature we were working on in a branch into *main*.

main has diverged enough from our branch that we want to integrate the new commits in it before merging our feature – so we merge *main* into our feature branch.

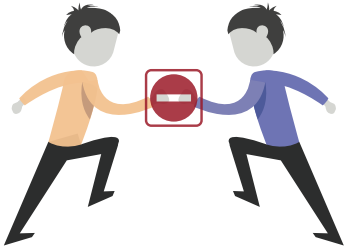
Conflicts

A merge conflict happens when trying to merge branches that have changes **in the same files and in the same lines**



Conflicts

```
(base) Matteos-MacBook-Pro-2:test matteotiberti$ git merge test
Auto-merging recipe.txt
CONFLICT (content): Merge conflict in recipe.txt
Automatic merge failed; fix conflicts and then commit the result.
```



Main branch Test branch

- Check which file(s) have conflicts.
- Modify files by hand to achieve the desired result.
- Add all files to stage once they are ready and commit. This solves the merge in a commit.
- **Remember to remove all conflict markers** and change your code in the final version you want.

```
- tomato
- mozzarella
- flour
- water
<<<<<<< HEAD
- capers
=======
- pineapple
>>>>>>> test
```

Exercise 3

Exercise
3

- 1. Create an ***experimental*** branch and switch to it. Change an ingredient in *ingredients.txt* and store the change as a commit. Try to get a list of the branches in the repository.
- 2. Go back to your ***main*** branch and add a step in *recipe.txt* and store the change as a commit. Check the difference between the two branches (use `git diff`).
- 3. Merge your ***experimental*** branch into your ***main*** branch. Check the history (try using the graph output).

Exercise 3 cont.

Exercise
3

- 4. Switch to your ***experimental*** branch and change the first line in one of your files, commit your changes.
- 5. Switch to your ***main*** branch and change the same first line in the same file in a different way, and commit.
- 6. Try to merge your ***experimental*** branch into your **main** branch; you should get a conflict. Check the status of the repository.
- 7. Edit the conflicting file to solve the conflict. Stage and commit to fix it. Check the history (try using the graph output).

Program

9.00-9.30: Theoretical introduction

9.30-10.15: First look at git/GitHub & commands

10.15-10.30: Coffee Break

10.30-11.15: Stage, commit & check history

11.15-12:00: Branches, merging & conflicts

12.00-13.00: Lunch break

13.00-13.30: Reset and revert (back to the past)

13.40-14.30: Interact with a remote repository

14.30-14.45: Coffee Break

14.45-15:30: Working together on GitHub

15.30-15:45: Git & RStudio / Code editors

15.45-16.00: Working with personal repos

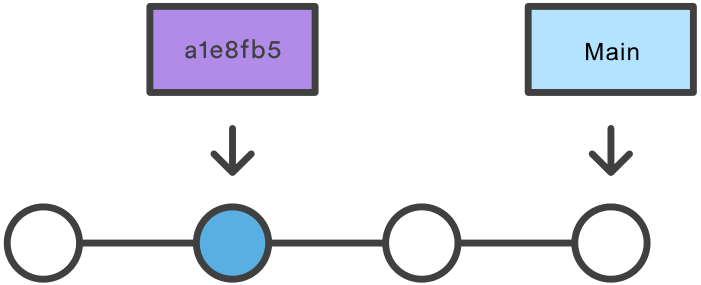
Back to the past

We can move around the repository using the command **`checkout`**

```
git checkout [identifier]
```

Changes the local content of the working directory to that of the reference - which means, moves **`HEAD`** to identifier. Identifier can be:

- A commit **ID**
- A branch **name**



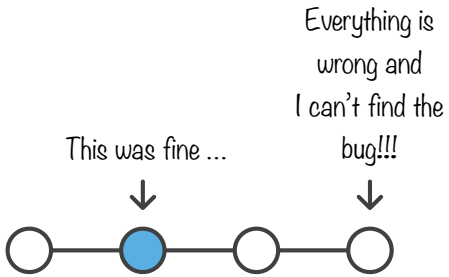
https://www.atlassian.com/git/tutorials/undoing-changes

Back to the past

Rewriting history is (usually) a bad idea

Every change you do in git is done with a forward in time perspective - the past should generally not be changed.

- Changes (even to something done a long time ago) should be done in commits added to the HEAD
- To restart from an earlier commit use branching/merging



I’ve botched staging or commit!

How to fix the last commit message:

```
git commit --amend -m “new message”
```

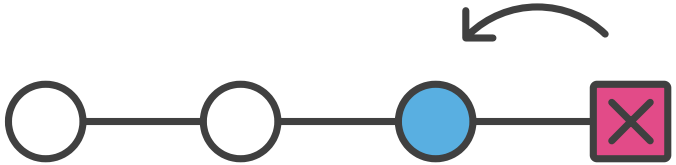
- Do not amend a message of a commit that have already been shared with others (e.g. uploaded to GitHub).

I’ve botched staging or commit!

`reset` is useful to “undo” some of the changes:

```
git reset
```

– Removes all staged changes.



```
git reset [filename]
```

– Removes file from staged area.

```
git reset --hard [ID]
```

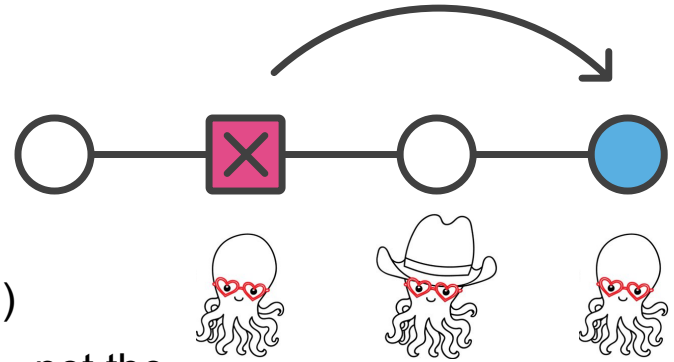
– Moves reference to that commit (can be HEAD) and resets the working directory to match that commit. **Destructive operation** - only on local branches that have not been shared with others yet **(do not use on *main*).**

https://www.atlassian.com/git/tutorials/undoing-changes/git-revert

I’ve botched staging or commit!

`revert` is used to add commits that “reverse” the changes done by older commits:

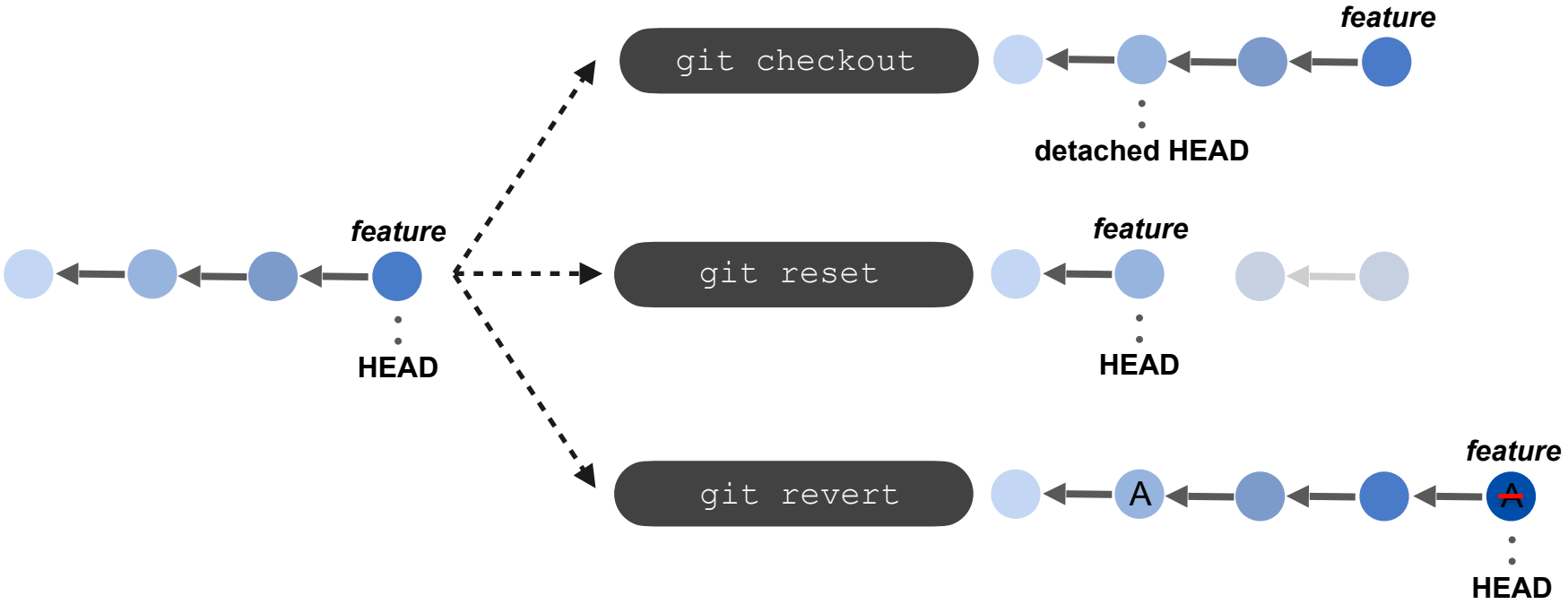
```
git revert [ID]
git revert [ID1]..[ID2]
```



- Here “ID” is the commit that you want to revert
- Non-destructive operation (history is maintained)
- History is polluted with commits that just revert – not the cleanest option
- Better than reset for shared repositories

https://www.atlassian.com/git/tutorials/undoing-changes/git-revert

Undoing changes



Exercise 4

Exercise
4

- 1. Checkout an earlier commit on the ***main*** branch. Check the content of the files. Check the history and status of the repository. What do you notice? Checkout ***main*** to get back to the tip of the branch.
- 2. Switch to the ***experimental*** branch. Add a step in *recipes.txt* and add the changes to stage, but **do not commit**. Check the status. Do a hard reset of **HEAD** and check the files and status. What do you notice?
- 3. Add a new ingredient to *ingredients.txt* and commit the changes. Add a new step to *recipe.txt* and commit the changes. Change the commit message of the last commit and check the history to see the change.
- 4. **Revert** the first of the two new commits (it will ask you to write a commit message). Check the files and history. What do you notice? Switch back to ***main***.

Program

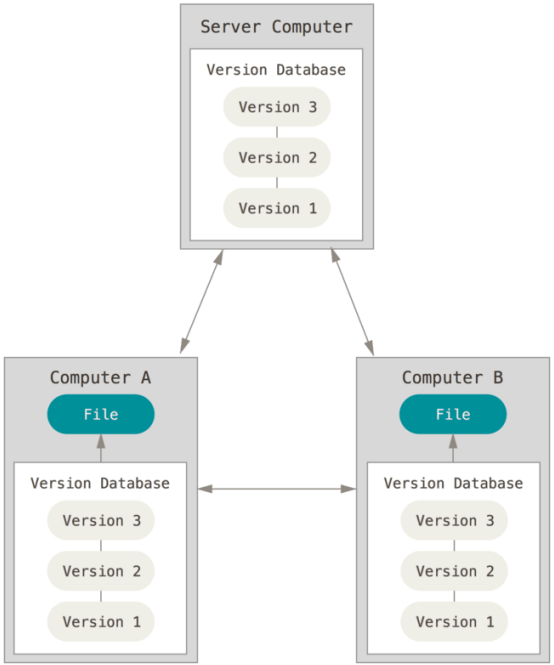
9.00-9.30:	Theoretical introduction
9.30-10.15:	First look at git/GitHub & commands
10.15-10.30:	Coffee Break
10.30-11.15:	Stage, commit & check history
11.15-12:00:	Branches, merging & conflicts
12.00-13.00:	Lunch break
13.00-13.30:	Reset and revert (back to the past)
13.40-14.30:	Interact with a remote repository
14.30-14.45:	Coffee Break
14.45-15:30:	Working together on GitHub
15.30-15:45:	Git & RStudio / Code editors
15.45-16.00:	Working with personal repos

Interacting with a remote

A **remote** is a repository that is linked to our local (here our local repository started out as a copy of a remote repository on GitHub).

Changes (commits) in the local repository can be uploaded and merged into the remote repository.

Changes that have happened in the remote repository (e.g. other people's) can be downloaded and merged into the local repository.



https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Personal Access Token (PAT)

https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens

Personal access token to use with the command line.

The token works as a password to the remote GitHub repository.

Advantage of PAT over password:

- **Unique**: Can be generated per use or per device.
- **Revokable**: Can revoke access to each one at any time.
- **Secure**: Random strings of characters that cannot be attacked by brute force i.e. less hackable than a password.
- **Quantity**: Any number of access tokens can be created, compared to only one password per user.

Interacting with a remote

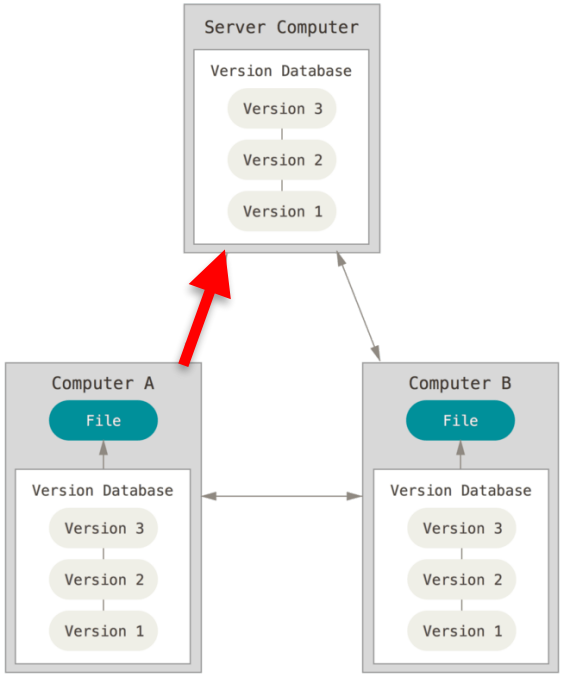
- Use **`push`** to upload local repository content to remote (i.e. export commits to remote branches)

```
git push [remote] [branch]
```

If this is your first time pushing to GitHub you need to login with your username and PAT.

- The default remote branch for the current local branch can be set when pushing (set upstream):

```
git push -u [remote] [branch]
```



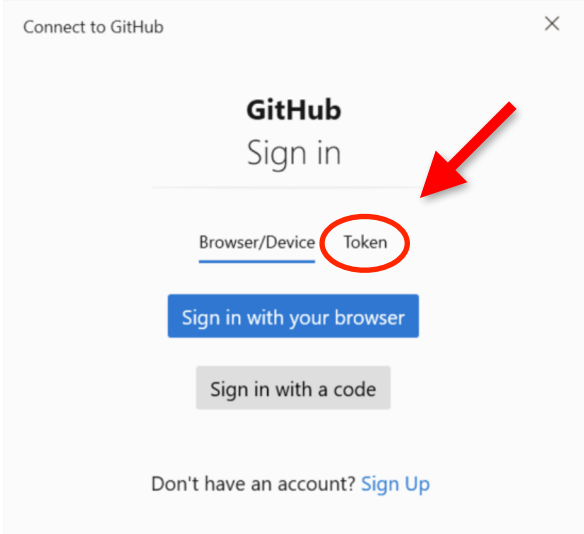
https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Interacting with a remote - Windows users

- Use **`push`** to upload local repository content to remote (i.e. export commits to remote branches)

```
git push [remote] [branch]
```

If this is your first time pushing to GitHub you need to login with your username and PAT.



- The default remote branch for the current local branch can be set when pushing (-- set upstream):

```
git push -u [remote] [branch]
```

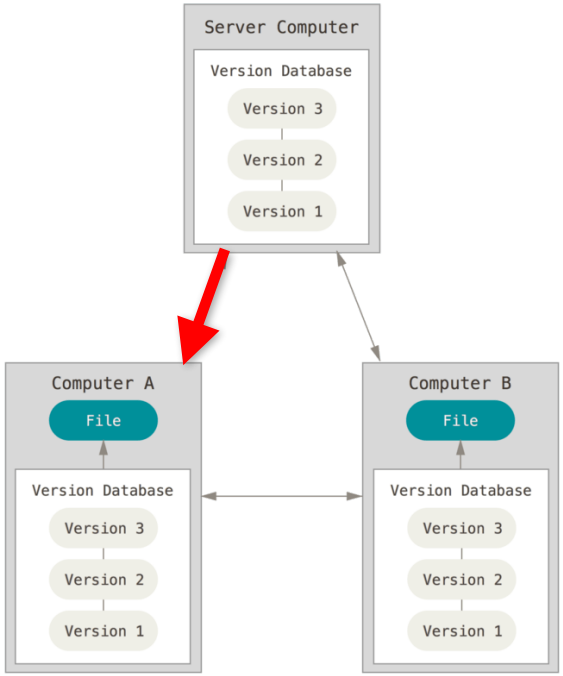
Interacting with a remote

- Use **`pull`** to download and apply the changes that were done in remote to your local repository (i.e. apply commits by other people):

```
git pull [remote] [branch]
```

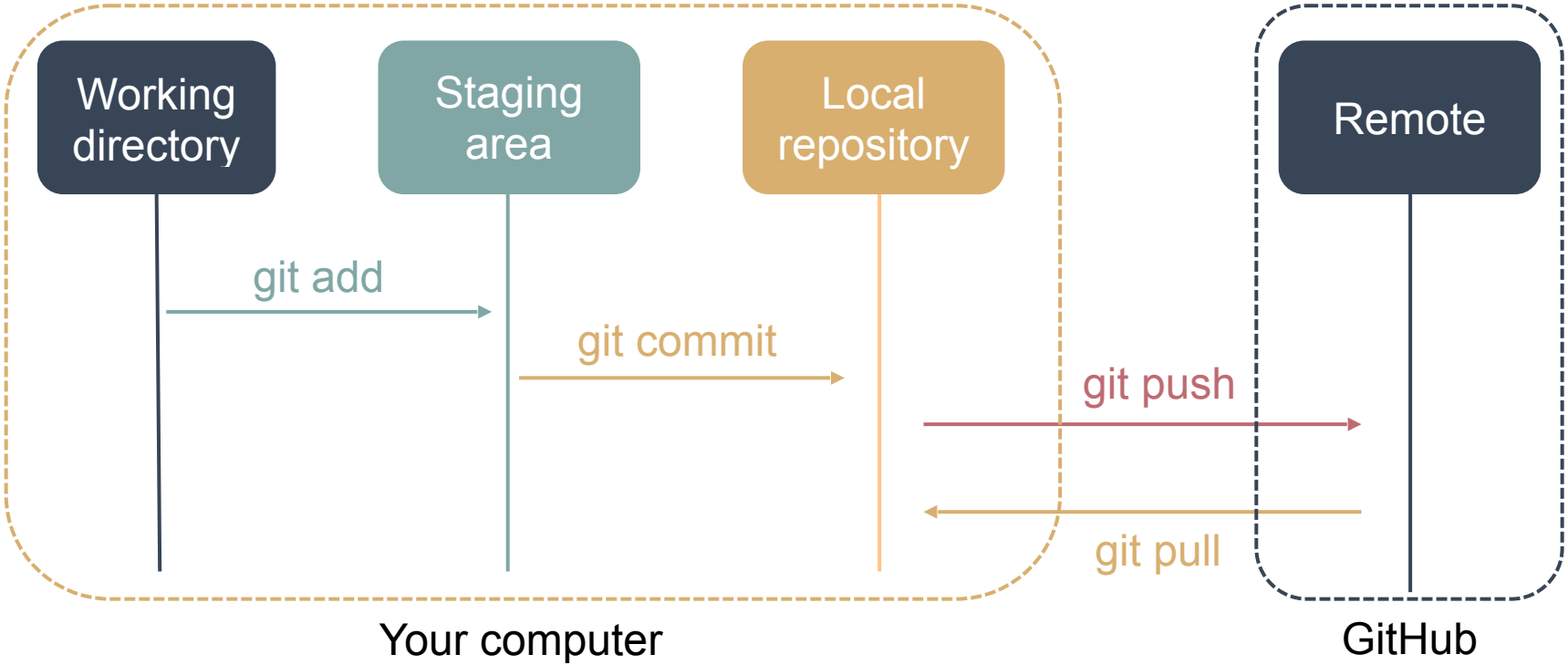
- Use **`fetch`** to download without merging (remote fetched branches are named as "`[remote]/[branch]`"):

```
git fetch [remote]
```



https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

What is staging



Exercise 5

Exercise
5

- 1. Go to your GitHub account and set up your personal access token (if you have not already done it). Follow the **Creating a personal access token (classic)** step-by-step on GitHub.com (click **repo** when selecting scopes)

- 2. Copy your access token and note your GitHub username as you will need both the first time you are pushing data to GitHub and for the RStudio part. If you have pushed to GitHub before you might not need to use it.

Exercise 5 cont.

Exercise
5

- 3. Push your changes to **your** remote repository for your ***main*** branch. Visit your remote repository on github.com. Check if your changes are there.
- 4. Push the ***experimental*** branch to your repository. Check the changes on GitHub. What do you notice?
- 5. Merge the ***experimental*** branch into ***main***. Push your changes and check them on GitHub. What has changed? Do you get any information about what has happened on GitHub?

Exercise 5 cont.

Exercise
5

- 6. Edit the **README.md file on GitHub**. Create a new commit (in your local repo) on **main** and try to push it. What happens?
- 7. **Fetch** the changes from the remote repository **(*origin)***. Note that the remote version of the ***main*** branch is named *origin/main.* Look at the difference between the local and remote version of ***main***.
- 8. Merge remote version ***origin/main*** into the local version of ***main*** and push the changes to the remote repository on GitHub. Visit GitHub and check if your changes are there.

Program

9.00-9.30: Theoretical introduction

9.30-10.15: First look at git/GitHub & commands

10.15-10.30: Coffee Break

10.30-11.15: Stage, commit & check history

11.15-12:00: Branches, merging & conflicts

12.00-13.00: Lunch break

13.00-13.30: Reset and revert (back to the past)

13.40-14.30: Interact with a remote repository

14.30-14.45: Coffee Break

14.45-15:30: Working together on GitHub

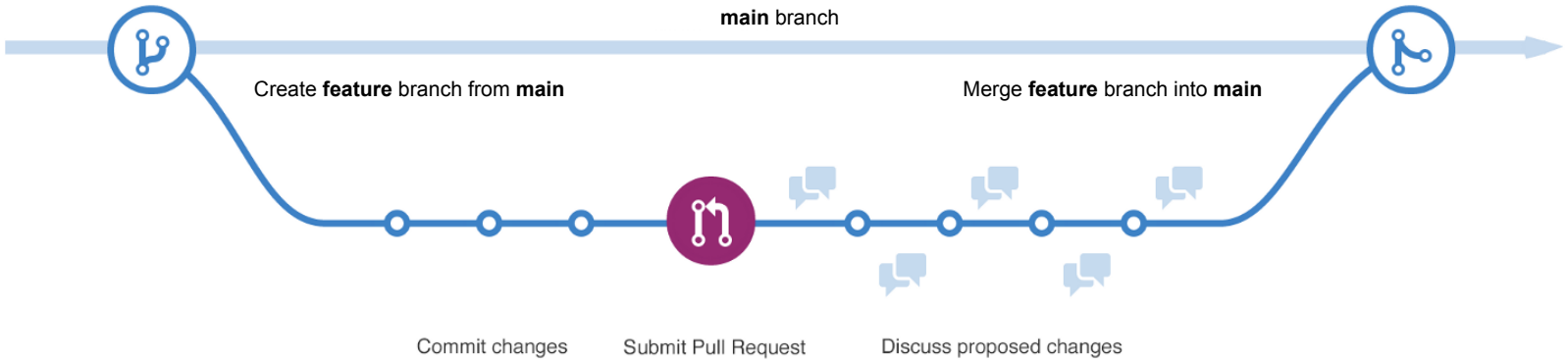
15.30-15:45: Git & RStudio / Code editors

15.45-16.00: Working with personal repos

Working together on GitHub

Contributing and interacting with other users

- Projects on GitHub often follow a pattern of development and communication.
- The *main* branch is considered the “stable” branch and is not changeable directly - only through **merges** and **pull requests**.
- Contributions happen in branches that are merged into *main*.
- Releases happens once in a while from the *main* branch.



Working together on GitHub

Interacting with other users: README

Read the documentation of the project you want to contribute to or use.

README.md

git-GitHub-workshop-2023

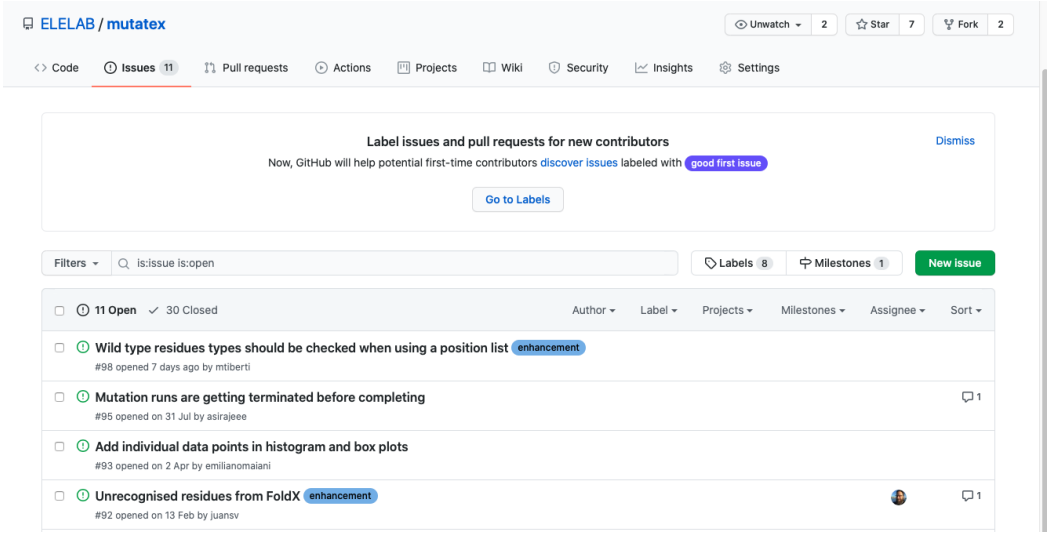
This repository is used for the HeaDS git and GitHub workshop 2023. Workshop participants should clone the repo after which they can use it to complete workshop exercises associated. The shared repo is made for hosting cooking recipies which are made locally by participents and then pushed to remote for everyone to view.

HeaDS workshop contacts:

Working together on GitHub

Interacting with other users: Issues

Check *Issues* if you have a problem or question to see if others have already asked the same. If needed, open a new issue.



Working together on GitHub

Interacting with other users: Branches

Create a local copy of the repository.

- **Clone** if you have write access to the repository (direct access as *collaborator* or part of the repo *organization*). Or if you are only planning on using the method and not contributing to its development.
- **Fork** if you have no affiliation to the repository or you want your own copy to experiment with.

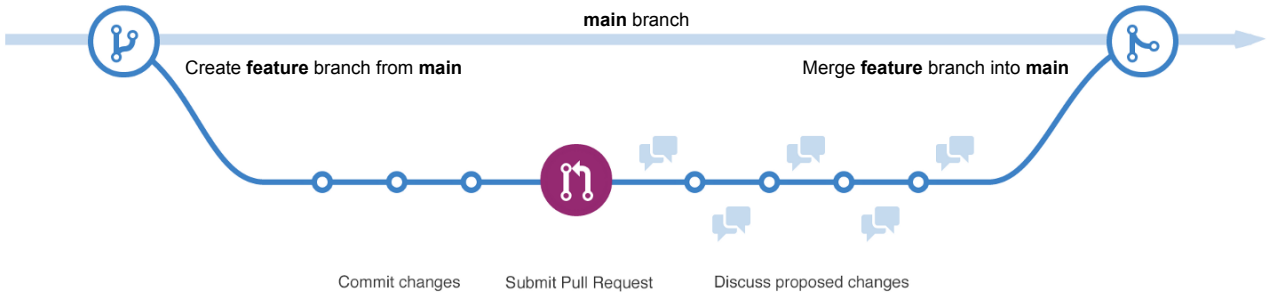
Create a **new branch** from the *main* branch. Add changes as commits. Push them so they are present on the remote (GitHub).

Working together on GitHub

Interacting with other users: Pull requests (PRs)

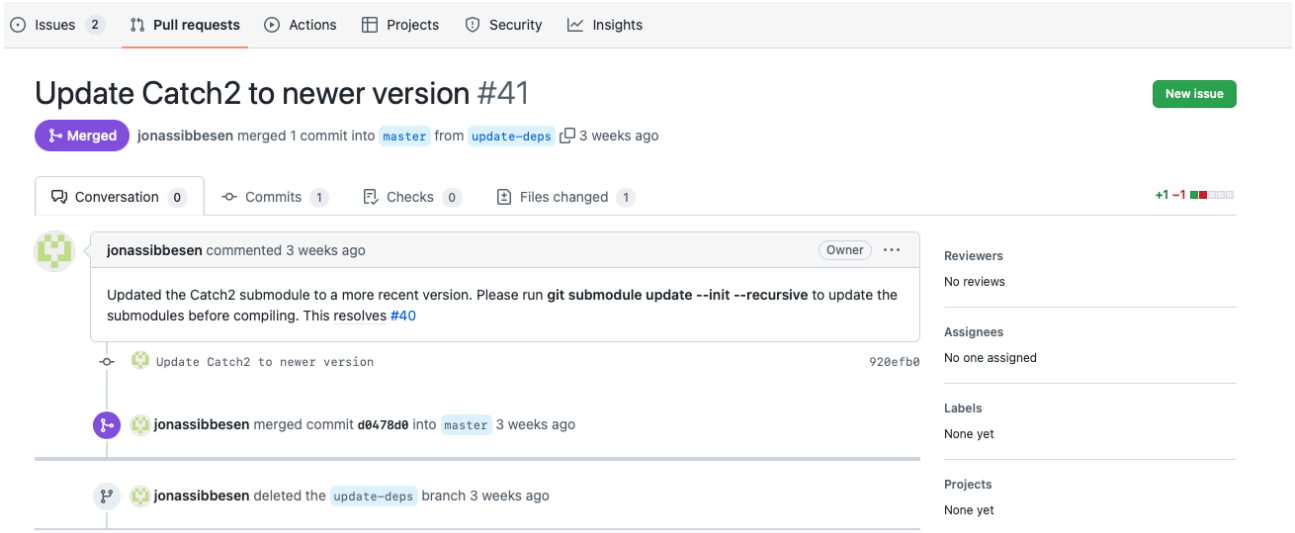
Create a *Pull Request (merge request)* from the new branch.

- Your code might be subject to code review. If so, the maintainers will decide if/when your code is good enough to be merged.
- To update the PR push the new changes to the same branch.
- Link to related issues in the PR. Use **keywords** (e.g. “resolves") to automatically close an issue when the PR is merged into *main*: “This PR resolves issue #40”.



Working together on GitHub

Interacting with other users: Pull requests (PRs)



Exercise 6

Exercise
6

- 1. Go to the shared recipes repository in GitHub for which you have become a collaborator.
- 2. Open an **issue** about a recipe you think is missing.
- 3. Open your terminal, make a local copy of the shared repository (where you just opened an issue) with **clone**.
- 4. Create a new branch. Switch to this branch.
- 5. Add your recipe to the new branch. Commit and push your changes (HINT: for pushing correctly recall what you did in exercise 5).

HINT: if you get an **error** a collaborator may have made changes to the **origin/main**, e.g. you have to **pull** first.

Exercise 6 cont.

Exercise
6

- 6. Go back to the GitHub repository (in the browser) and open a **Pull request (compare and pull)** with your new branch.
- 7. In your **pull request** message link to your **issue** with one of the **keywords** listed **here**
- 8. Wait for one of us to approve the PR. Pull from the repository and check our shared recipe book.
- 9. Your **Pull request** may be rejected with comments for changes. **Implement** the changes and open a **new Pull request**.

Program

9.00-9.30: Theoretical introduction

9.30-10.15: First look at git/GitHub & commands

10.15-10.30: Coffee Break

10.30-11.15: Stage, commit & check history

11.15-12:00: Branches, merging & conflicts

12.00-13.00: Lunch break

13.00-13.30: Reset and revert (back to the past)

13.40-14.30: Interact with a remote repository

14.30-14.45: Coffee Break

14.45-15:30: Working together on GitHub

15.30-15:45: Git & RStudio / Code editors

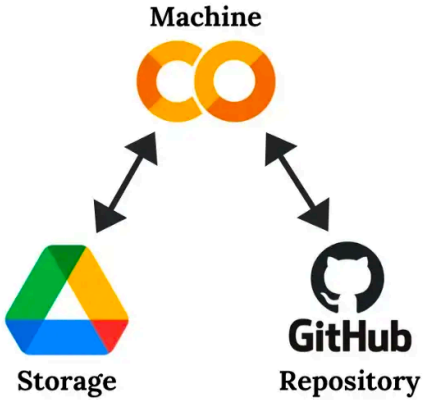
15.45-16.00: Working with personal repos

Git & GitHub with Code Editors

- You can work with **git/GitHub** from within code-editors, including Rstudio (R), Colab / Jupyter Notebook, Visual Studio, etc.

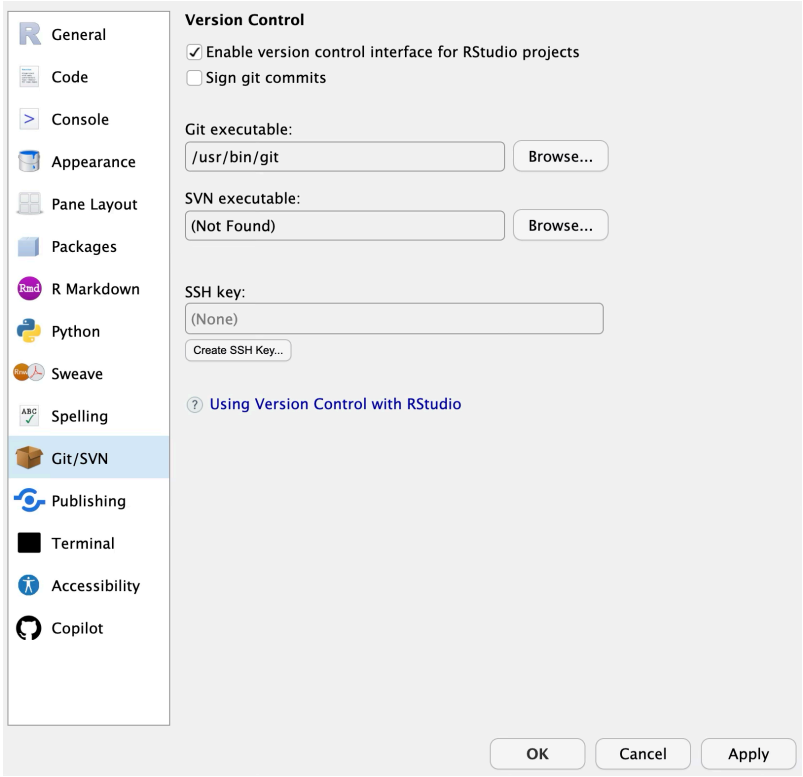
Rstudio

Visual Studio Code

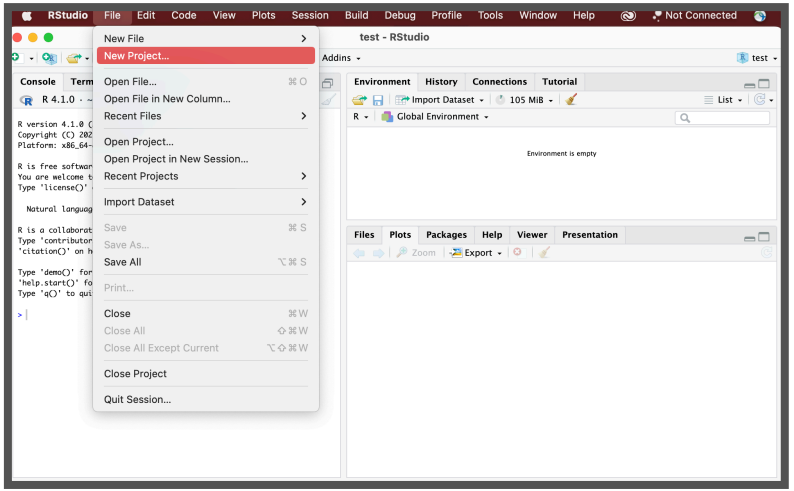


Configure RStudio

- **Make Sure RStudio Knows about Git**
- Go to the Edit → Settings → Git/SVN
- Enable the version control interface and make sure RStudio knows where to find git.



Git & RStudio

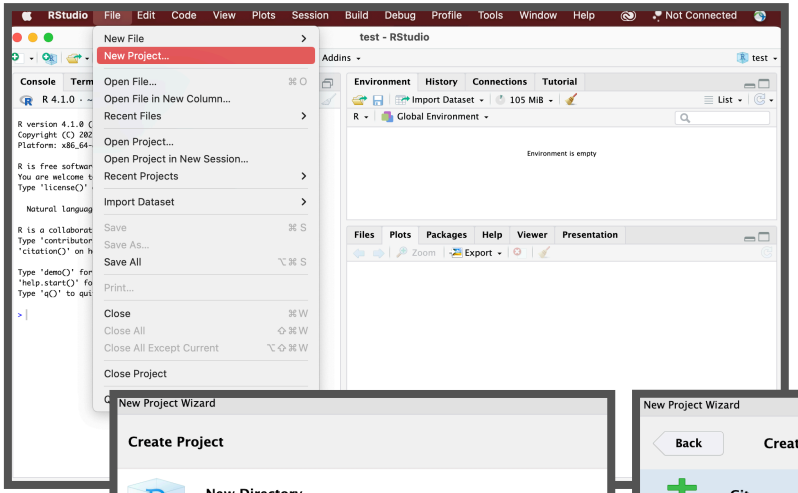


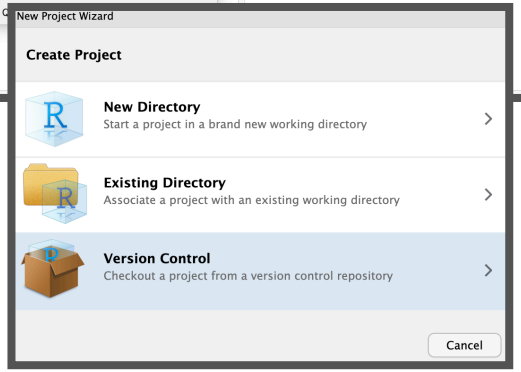
Researchers working with R benefit from using version control.

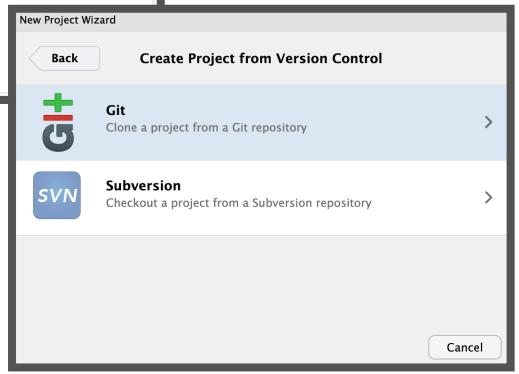
From within RStudio Project!

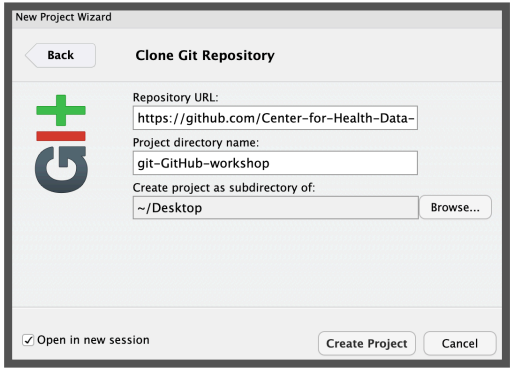
Notice where the Project will be saved locally!

Git & RStudio - GitHub Project





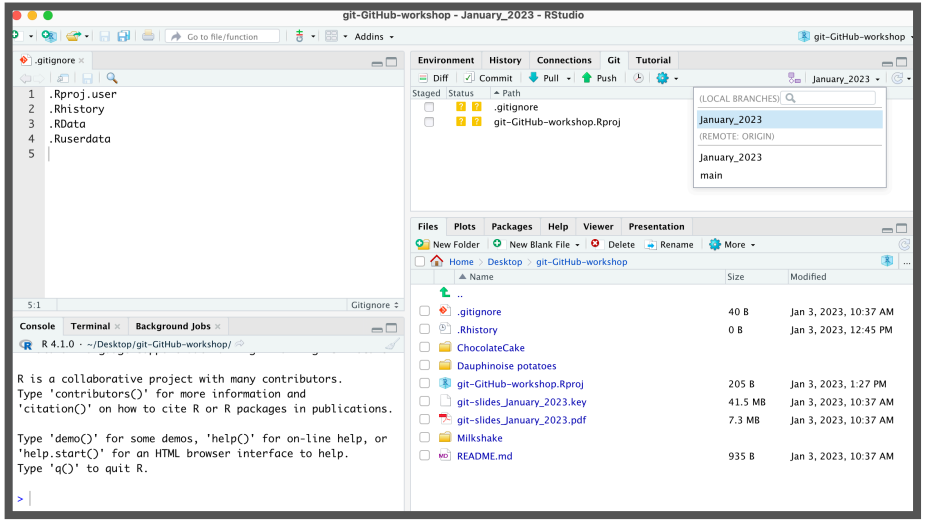


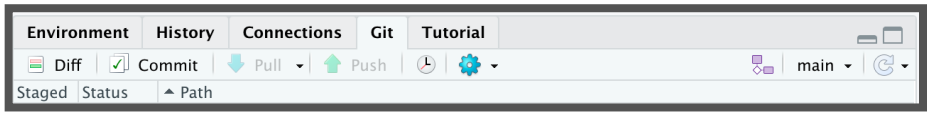


New R project from the GitHub repository

- Copy the GitHub repository url
- In Rstudio, File > New Project..., select Version Control, choose Git
- Provide the repository url link you just copied

Git & RStudio



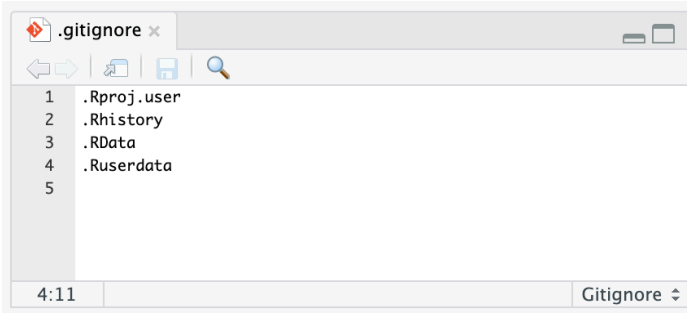


RStudio's git panel contains a **subset of commands** from **git**

More complex commands from the command line

- Make local changes, stage and commit
- Create and switch between branches
- Changes to scripts can be reverted, and pushed to remote (e.g. GitHub)
- Click the green "Push" button to send your local changes to GitHub.

Ignore files from version control



When starting a new project in RStudio, you initialize a .gitignore file (if it does not already exists).

File or folder names in the .gitignore will not be taken into account by git or sync to GitHub

Examples of files you would potentially want to ignore:

- Sensitive information (passwords,…)!
- Binary files. Git works very well with text files, but not with binary files such as .Rdata.
- Files > 50MB. Git is specifically made for code (e.g. .R) and does not intend to track all changes in large data files (though there are options for this).
- A temp/ folder inside your project with ‘disposable’ content, e.g. draft, test or temp.

Configure Git

- You want git to know who you are so it can associate your changes with you. If you haven't configured Git already you can do it from within RStudio.
- Via the usethis package in R:

```
> install.packages("usethis")
> library(usethis)
> usethis::use_git_config(user.name="Jane Doe", user.email="jane@example.org")
```

- edit_git_config() function will open your gitconfig file. Add your name and email and close this.

```
> edit_git_config()
```

Personal Access Token & RStudio

- How to Connect RStudio Projects with GitHub Repository?
- **Your Personal Access Token** will need to be stored in a way RStudio can access it to connect to your GitHub account.

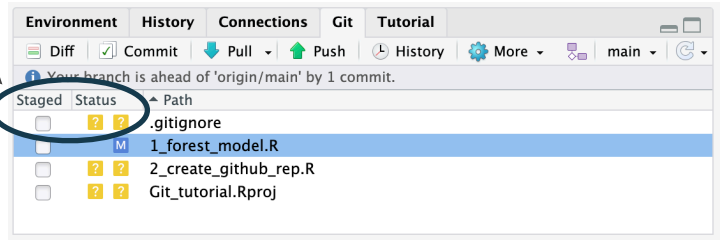
- There are a couple of ways to get your PAT into the Git credential store:
    - Call an R function to explicitly store (or update) your credentials.
    - Do something in RStudio that triggers a credential challenge.

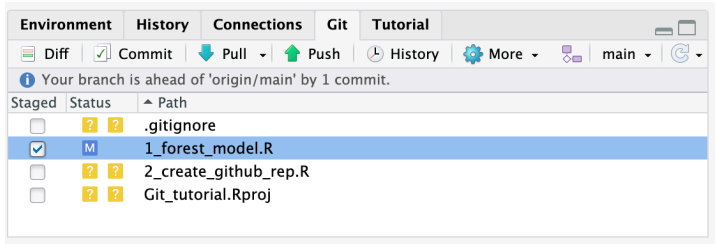
Personal Access Token & RStudio

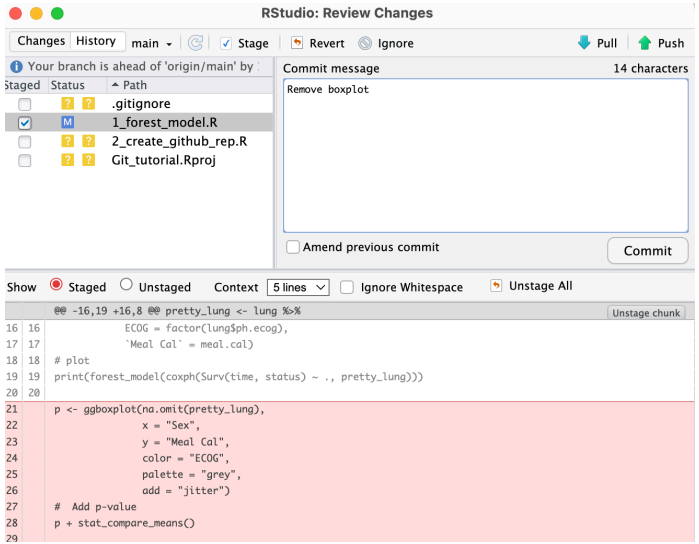
- **"gitcreds" package.** If you've installed "usethis", you have gitcreds
- ***gitcreds::gitcreds_set().*** If you don't have a PAT stored already, it will prompt you to enter your PAT. Paste!
- If you already have a stored credential, it will let you inspect it.
- You'll enter your GitHub username and the Personal Access Token as your password

```
> install.packages("gitcreds")
> library(gitcreds)
> gitcreds_get()
#You can check that you've stored a
#credential with gitcreds_get():

> gitcreds::gitcreds_set()

? Enter password or token:
ghp_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxx
-> Adding new credentials...
-> Removing credentials from
cache...
-> Done.
```

Stage and Commit







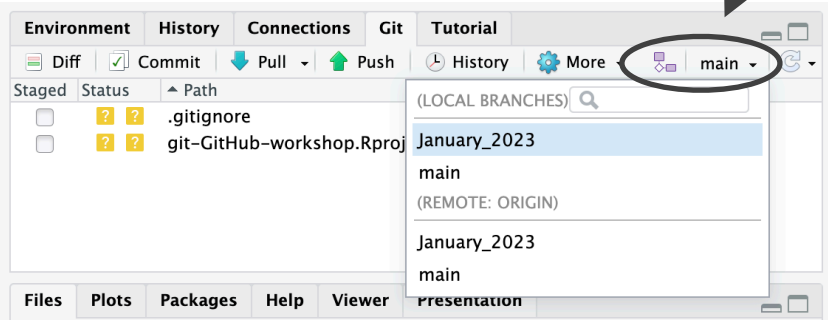
- Added
- Deleted
- Modified
- Renamed
- Untracked

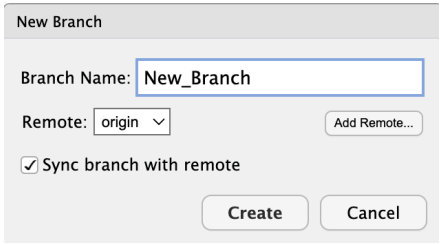
The status of the new files/changes

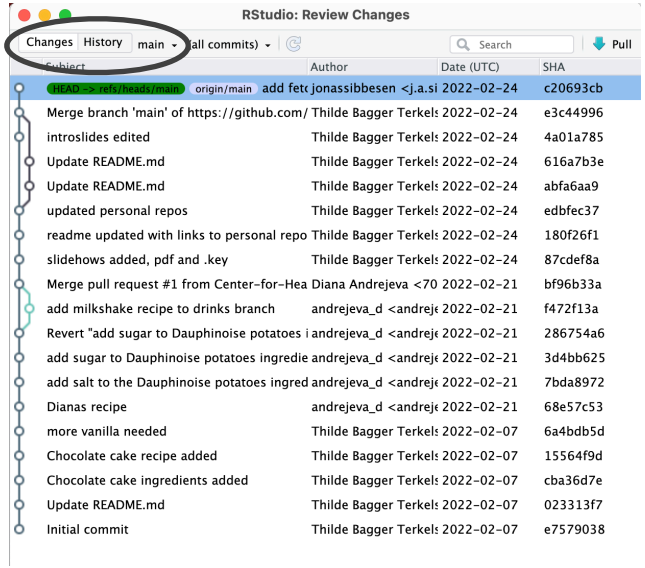
By checking the box and clicking commit, we add the file and are able to commit this with a commit message.

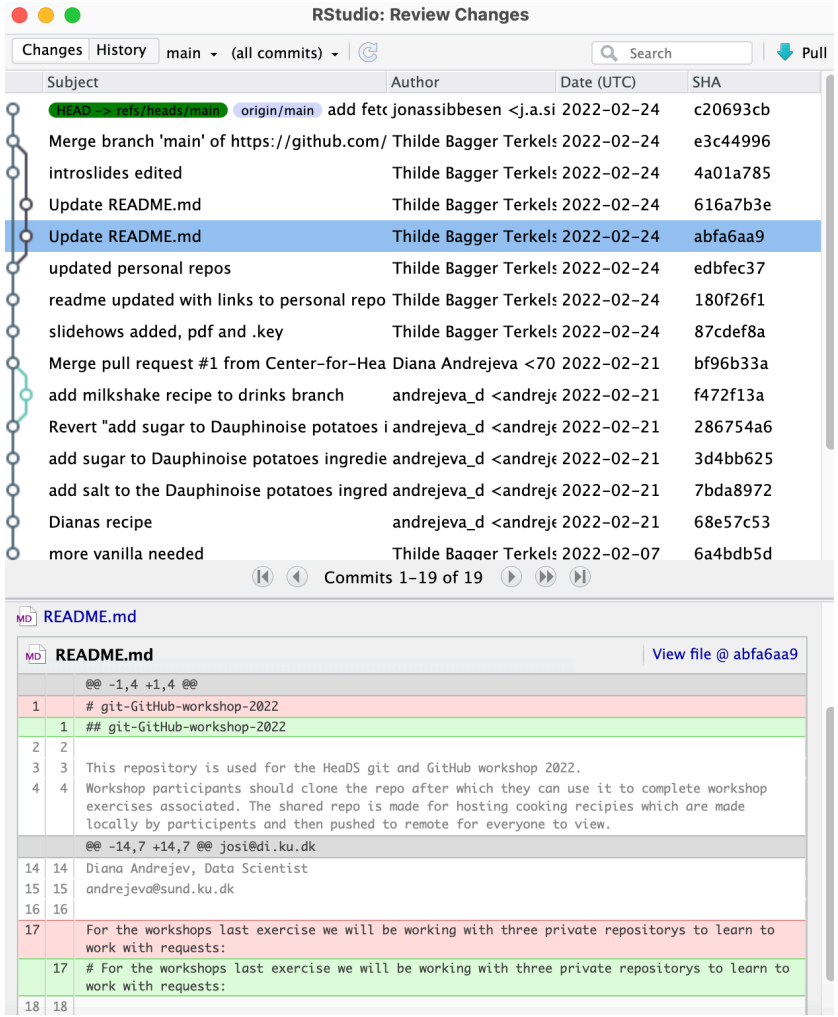
THEN finally we push to GitHub!

Branching

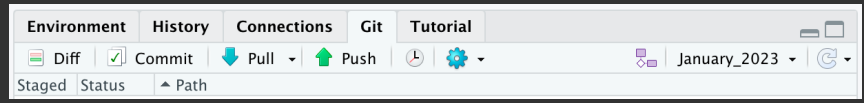

Environment
History
Connections
Git
Tutorial
Diff
Commit
Pull
Push
History
More
main
Staged
Status
Path
.gitignore
git-GitHub-workshop.Rproj
(LOCAL BRANCHES)
January_2023
main
(REMOTE: ORIGIN)
January_2023
main
Files
Plots
Packages
Help
Viewer
Presentation


New Branch
Branch Name:
New_Branch
Remote:
origin
Add Remote...
Sync branch with remote
Create
Cancel


RStudio: Review Changes
Changes
History
main
all commits)
Search
Pull
Author
Date (UTC)
SHA
HEAD -> refs/heads/main
origin/main
add fetc
jonassibbesen <j.a.si
2022-02-24
c20693cb
Merge branch 'main' of https://github.com/
Thilde Bagger Terkels
2022-02-24
e3c44996
introslides edited
Thilde Bagger Terkels
2022-02-24
4a01a785
Update README.md
Thilde Bagger Terkels
2022-02-24
616a7b3e
Update README.md
Thilde Bagger Terkels
2022-02-24
abfa6aa9
updated personal repos
Thilde Bagger Terkels
2022-02-24
edbfec37
readme updated with links to personal repo
Thilde Bagger Terkels
2022-02-24
180f26f1
slidehows added, pdf and .key
Thilde Bagger Terkels
2022-02-24
87cdef8a
Merge pull request #1 from Center-for-Hea
Diana Andrejeva <70
2022-02-21
bf96b33a
add milkshake recipe to drinks branch
andrejeva_d <andrej
2022-02-21
f472f13a
Revert "add sugar to Dauphinoise potatoes
andrejeva_d <andrej
2022-02-21
286754a6
add sugar to Dauphinoise potatoes ingredie
andrejeva_d <andrej
2022-02-21
3d4bb625
add salt to the Dauphinoise potatoes ingred
andrejeva_d <andrej
2022-02-21
7bda8972
Dianas recipe
andrejeva_d <andrej
2022-02-21
68e57c53
more vanilla needed
Thilde Bagger Terkels
2022-02-07
6a4bdb5d
Chocolate cake recipe added
Thilde Bagger Terkels
2022-02-07
15564f9d
Chocolate cake ingredients added
Thilde Bagger Terkels
2022-02-07
cba36d7e
Update README.md
Thilde Bagger Terkels
2022-02-07
023313f7
Initial commit
Thilde Bagger Terkels
2022-02-07
e7579038



History

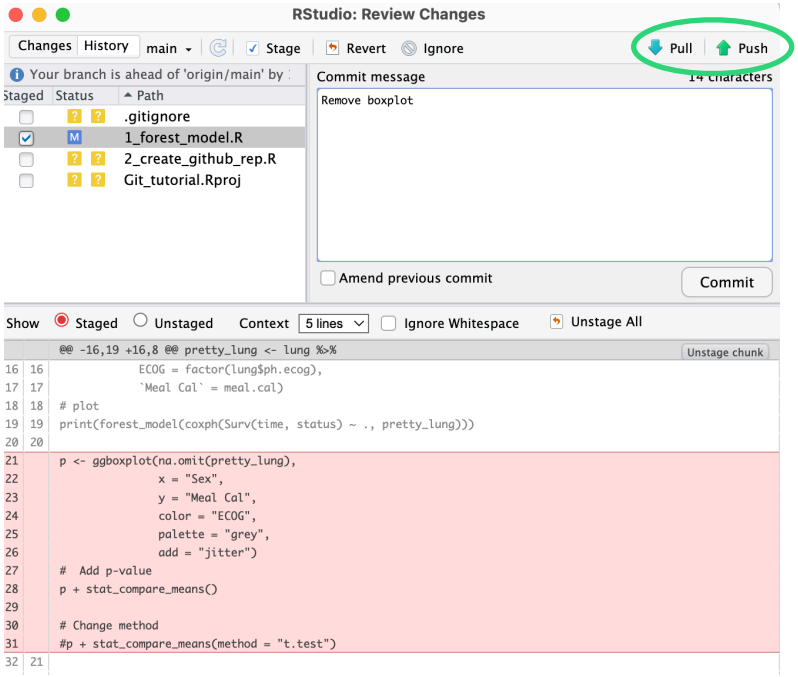


- Notice the presence of History twice:
- Git history vs R command history

- You can check the history online, using the terminal as well as **in Rstudio**.

- Click on history in the **Git panel**

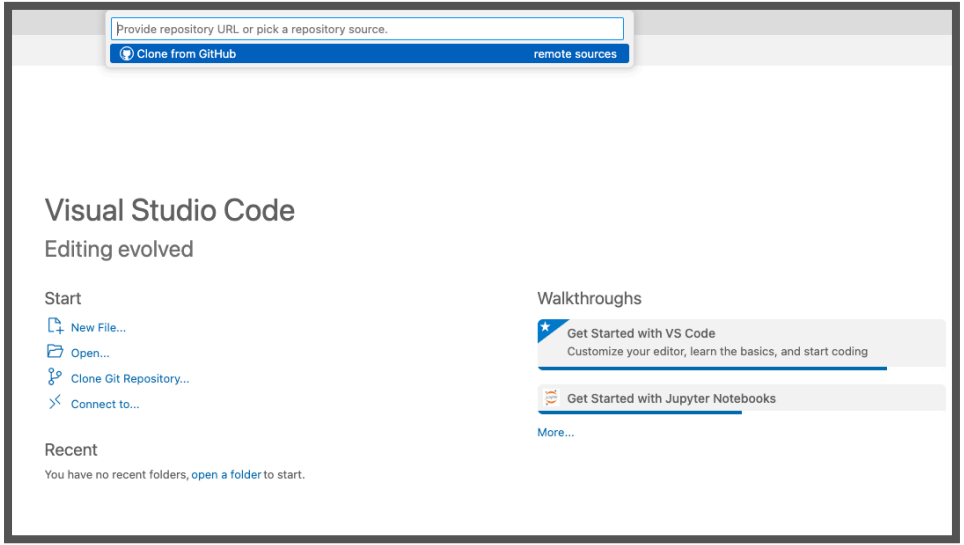
Interact with GitHub



- **Pull updates from GitHub**
  - Down arrow button, go to GitHub, grab the most recent code and bring it into local editor.
  - Pulling regularly is important!
- **Push changes to GitHub**
  - Rstudio gives a warning about the status of your local commits vs those stored on GitHub.
  - After committing, we have a push button (the up arrow) in RStudio.

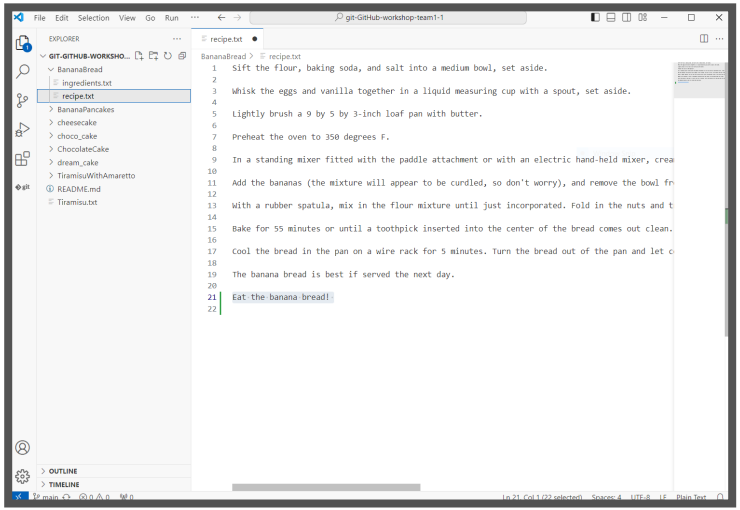
Visual Studio Code

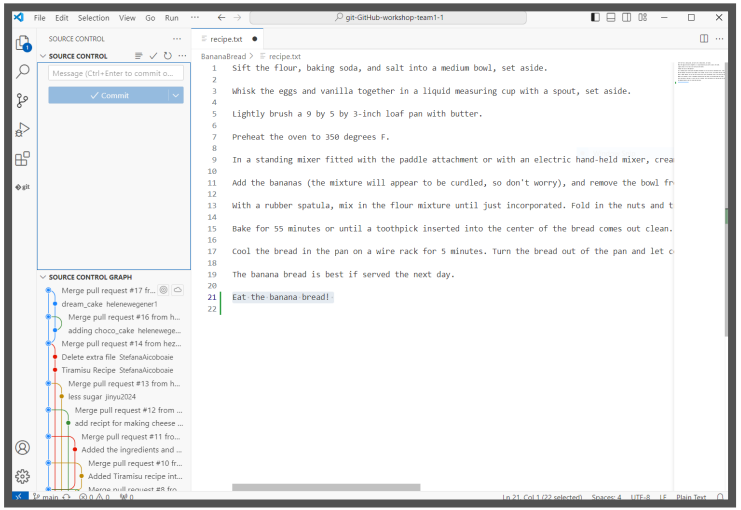
- Very easy to use VS code with git
- When you open a new session you can clone a remote repo with URL
- .. or initialise a new local git repo



Visual Studio Code

- Use **add**, **commit** (message) and **push** button
- See **git log history** of commits

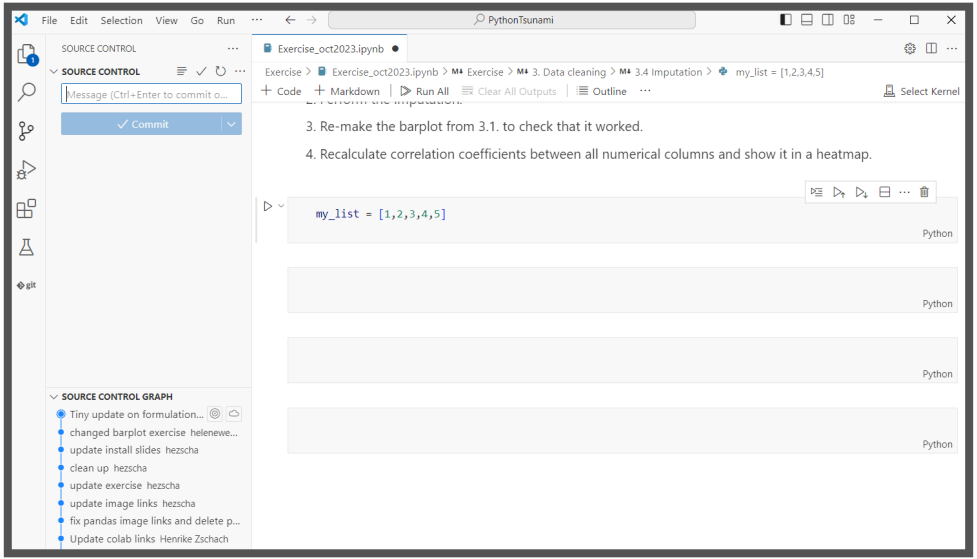




Visual Studio Code

- Works with Jupyter Notebooks
- … and Conda environments

- Prerequisites:
  - Code language installed
  - Notebooks installed
  - Git installed



N.B IF you have never used git and launch it first in VS code, you will be asked for git user and token the first time.

Git & RStudio

Exercise

7

- Go to Github and create a new repository. Name it whatever you'd like. Add a ***readme*** file, .***gitignore*** file and ***license*** - see **slide 27** (*Let's create a GitHub repo*)
- Open Rstudio on your local computer. Make an R git version of your new personal repository - see **slide 99** (*Git & RStudio - GitHub Project*)
- Edit your ***readme*** file in Rstudio and **make an R-script** with some content.
- Add and commit your changes, and push them to the remote. Check that they are there.

N.B if you have issues, follow slides 99-104 on setting up git correctly for Rstudio.

Git & VScode

Exercise

7

- Go to Github and create a new repository. Name it whatever you'd like. Add a ***readme*** file, .***gitignore*** file and ***license*** - see **slide 27** (*Let's create a GitHub repo*)
- **Open VS** code on your local computer. **Clone** your git repo via the **command palette** (command + shift + p).
- Edit your ***readme*** file in VS code and **make a script** (R or Python) **or a notebook** (Quarto/Jupyter) with some content.
- Add and commit your changes, and push them to the remote. Check that they are there.

N.B IF you are lacking prerequisites we recommend a global installation of these!

Thank you for today

See you at HeaDS